

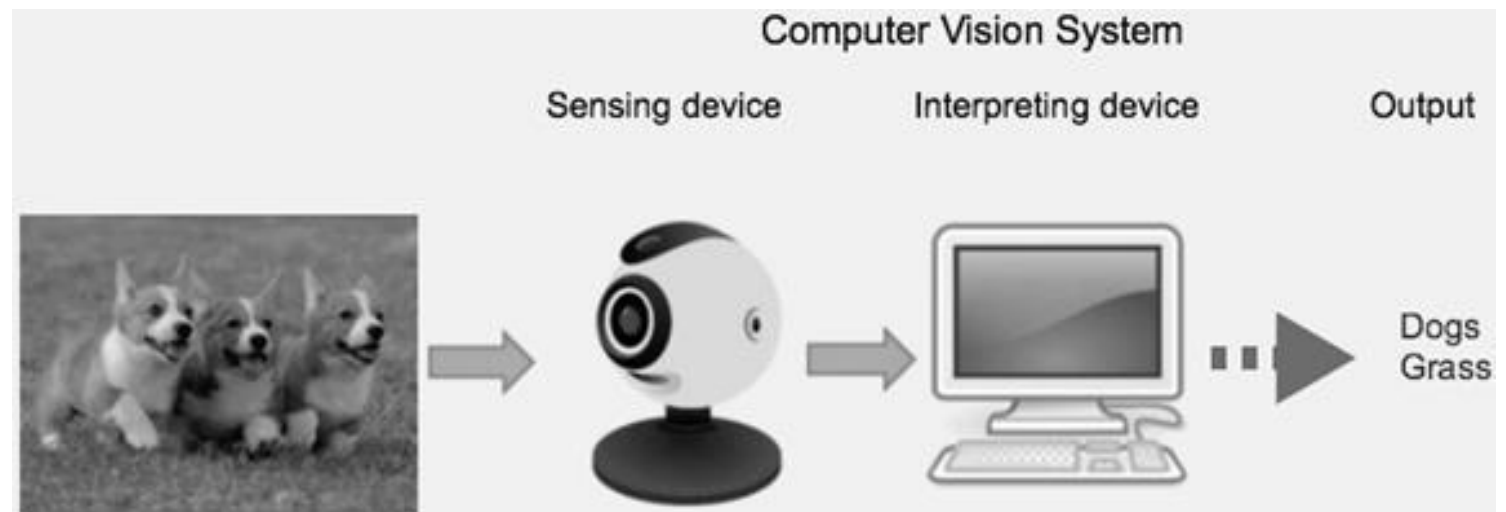
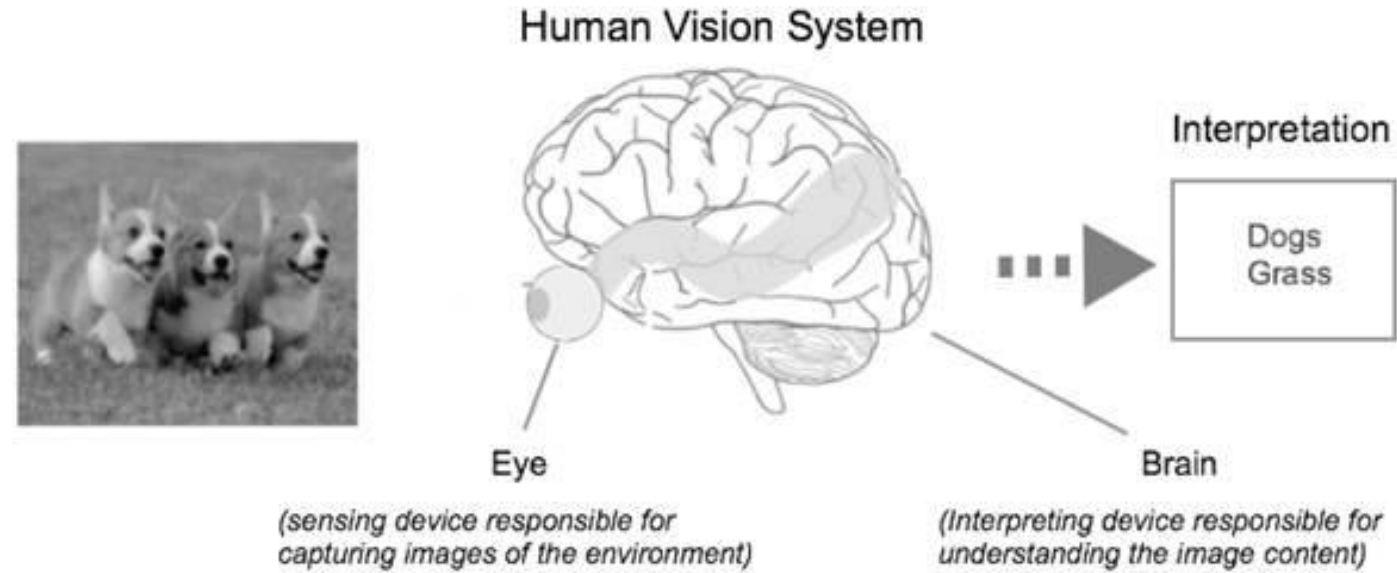
Deep learning in computer vision

Dr. Praisan Padungweang, Collage of Computing, KKU

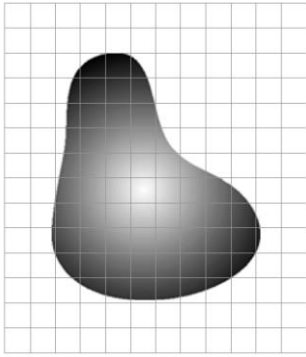
Credit: Steve Elston



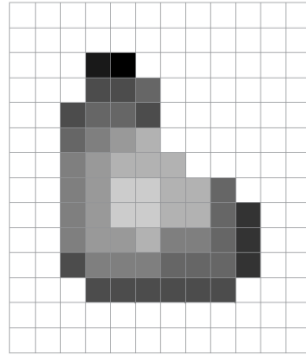
Human Vision Systems vs AI Vision Systems



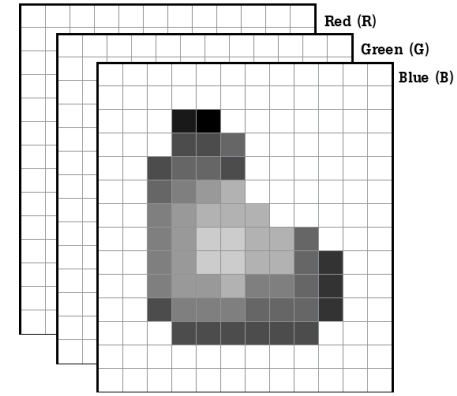
Digital image



Real Object



gray scale



RGB image

RGB Channel in color image



Color image



Red channel

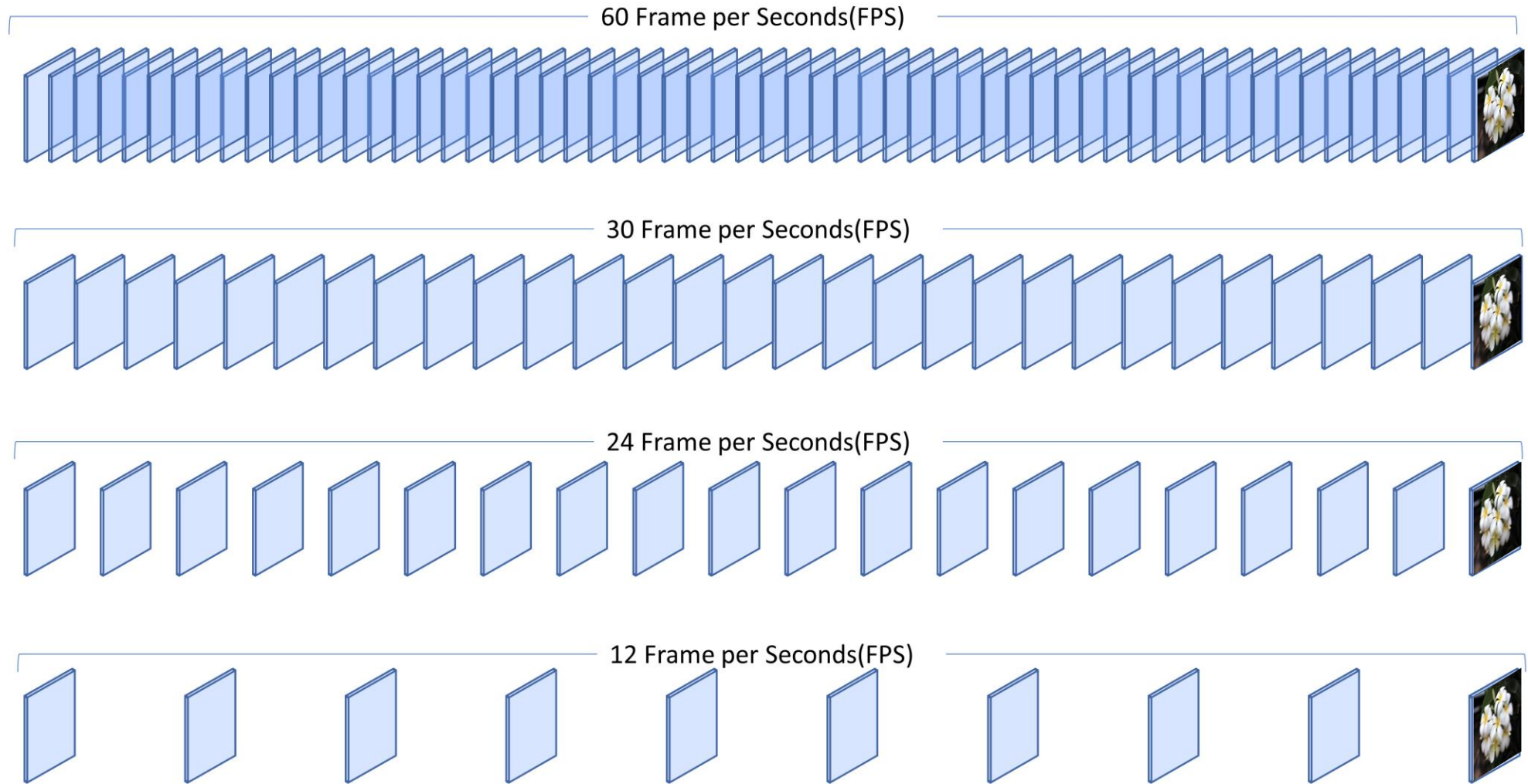


Green channel



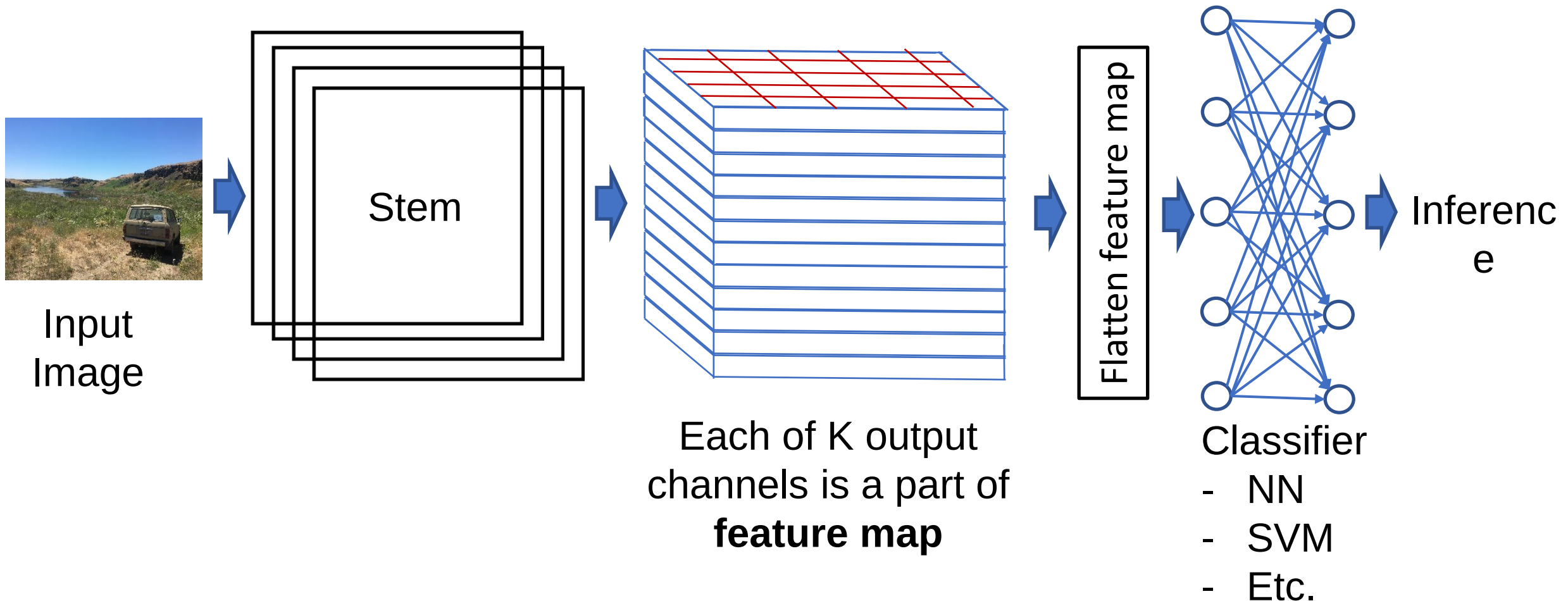
Blue channel

Video



Convolutional Neural Networks

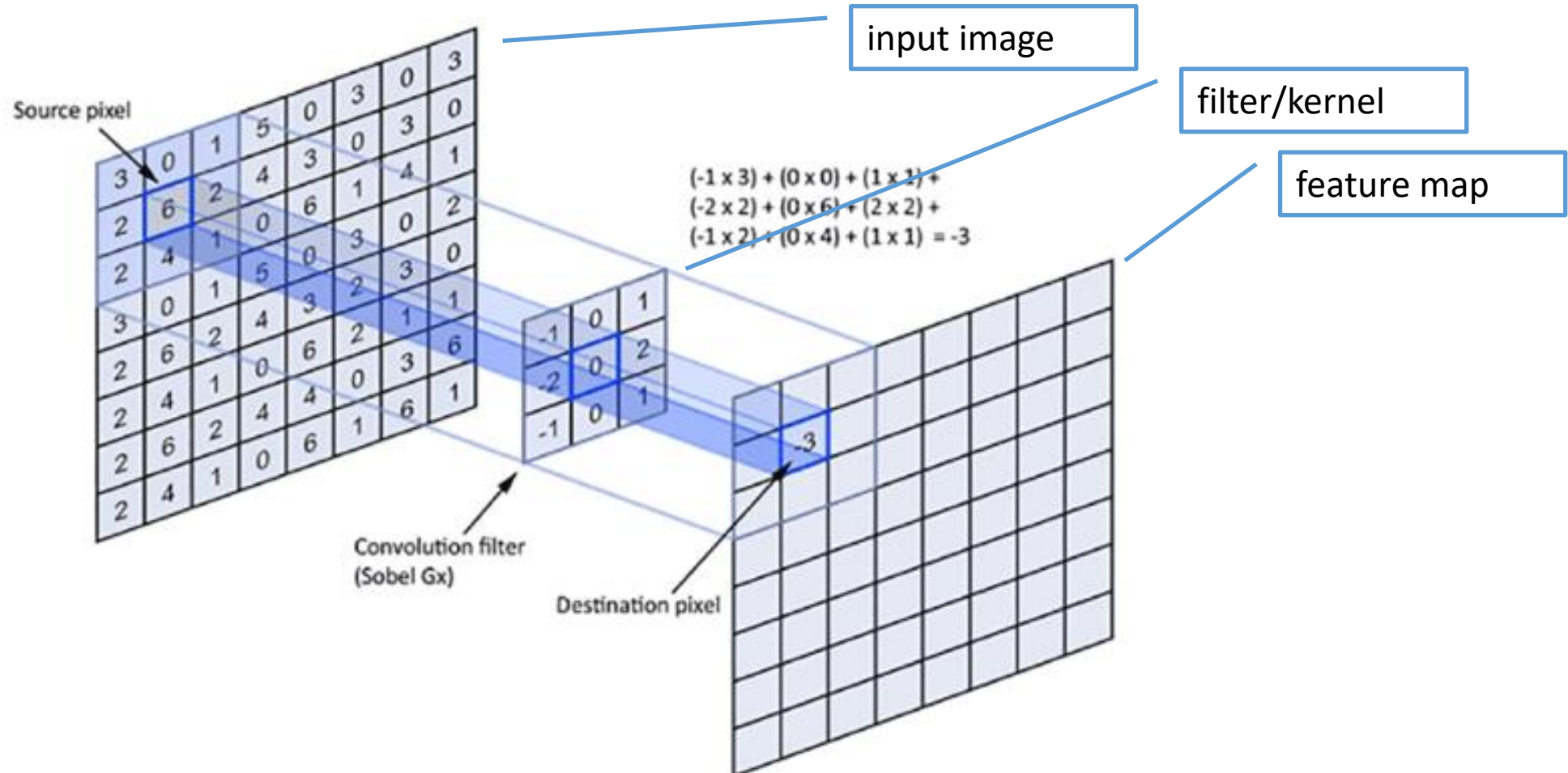
CNN creates feature map in inference pipeline



Convolution operation

Creates feature maps highlighting patterns

1. Slide a filter/kernel over the input image
2. At each position, perform element-wise multiplication and sum



Convolution operation

Example

$$\begin{aligned}
 &1 \cdot (-2) + 8 \cdot 1 + 15 \cdot 2 + \\
 &7 \cdot (-2) + 14 \cdot 3 + 16 \cdot 2 + \\
 &13 \cdot (-2) + 20 \cdot (-2) + 22 \cdot 2 = 74
 \end{aligned}$$

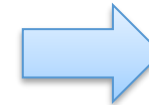
image

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

⊗

kernel

-2	1	2
-2	3	2
-2	-2	2



-19	10	74
3	72	36
70	34	33

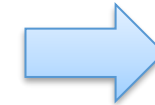
Convolution operation: Zero Padding

- Zero padding refers to the process of symmetrically adding zeroes to the input matrix.

0	0	0	0	0	0	0
0	17	24	1	8	15	0
0	23	5	7	14	16	0
0	4	6	13	20	22	0
0	10	12	19	21	3	0
0	11	18	25	2	9	0
0	0	0	0	0	0	0

⊗

-2	1	2
-2	3	2
-2	-2	2

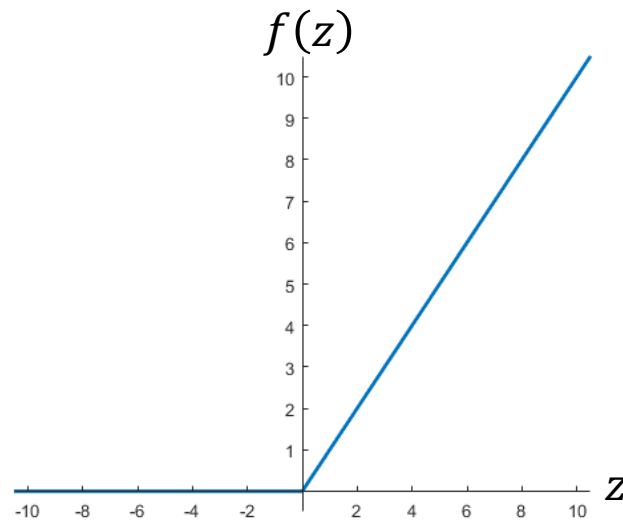


63	-2	-25	42	-31
148	-19	10	74	-65
61	3	72	36	-34
84	70	34	33	-73
103	112	80	-37	-16

Activation function: Rectified Linear Unit (ReLU)

$$f(z) = \begin{cases} z, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

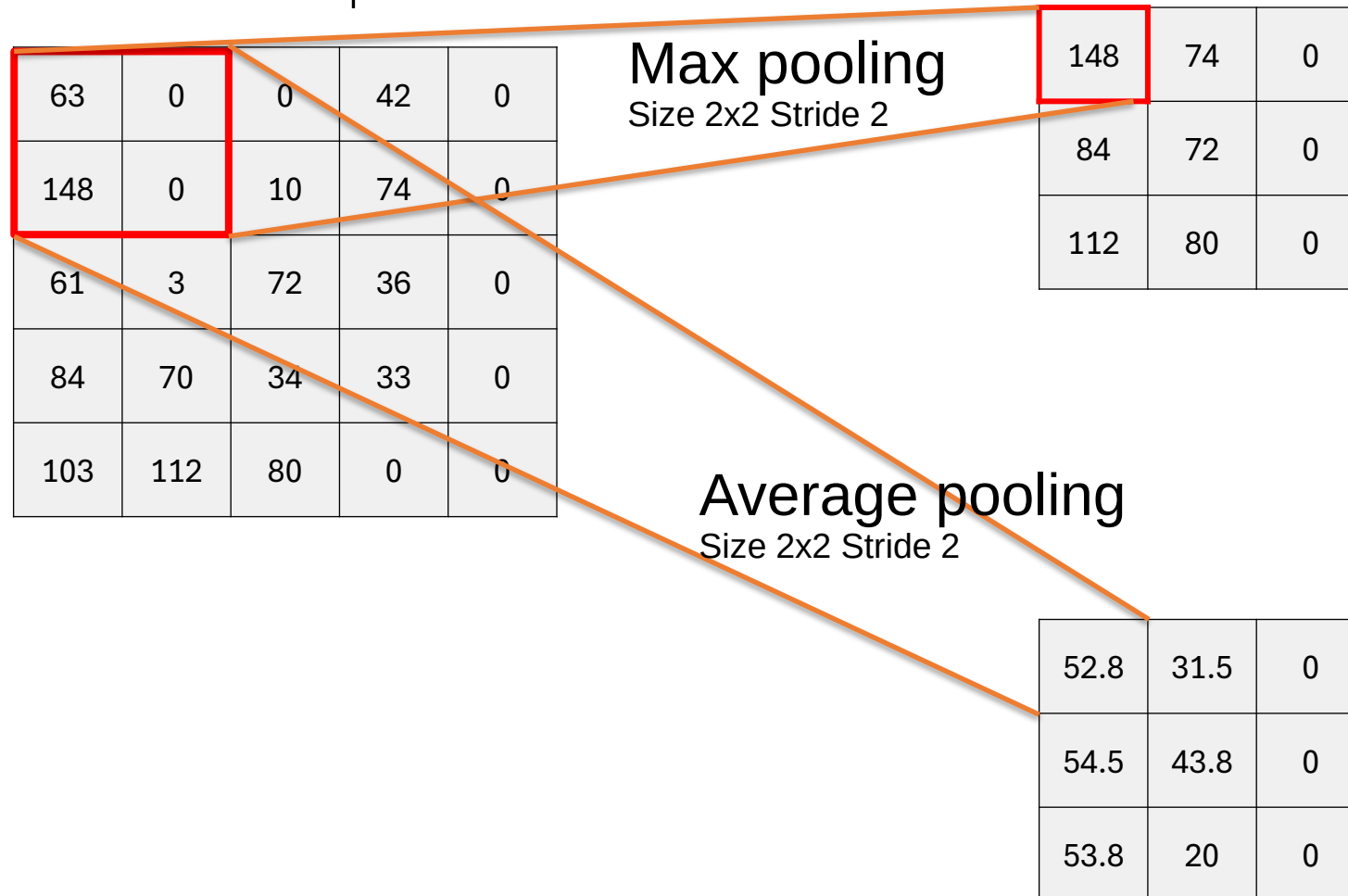
63	-2	-25	42	-31
148	-19	10	74	-65
61	3	72	36	-34
84	70	34	33	-73
103	112	80	-37	-16



63	0	0	42	0
148	0	10	74	0
61	3	72	36	0
84	70	34	33	0
103	112	80	0	0

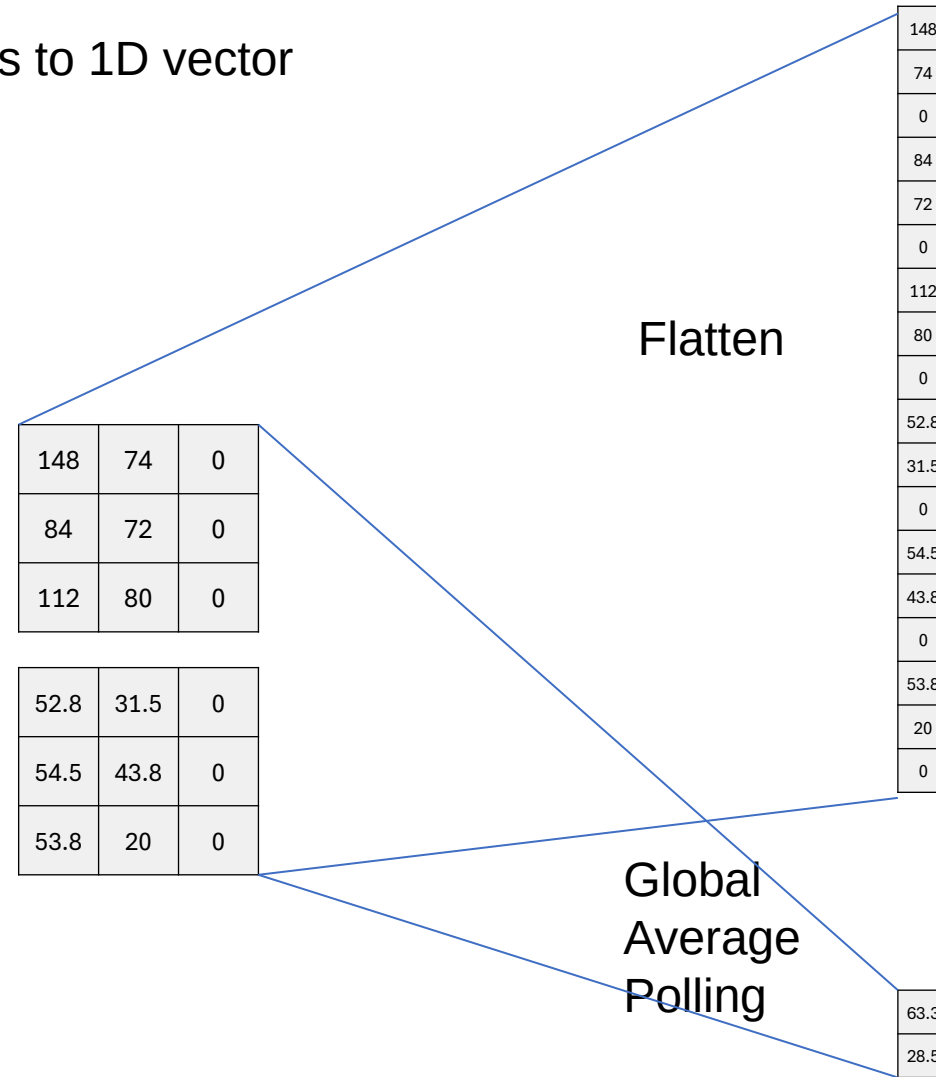
Pooling

Pooling layers perform downsampling operations to reduce the spatial dimensions (width and height) of the feature maps.

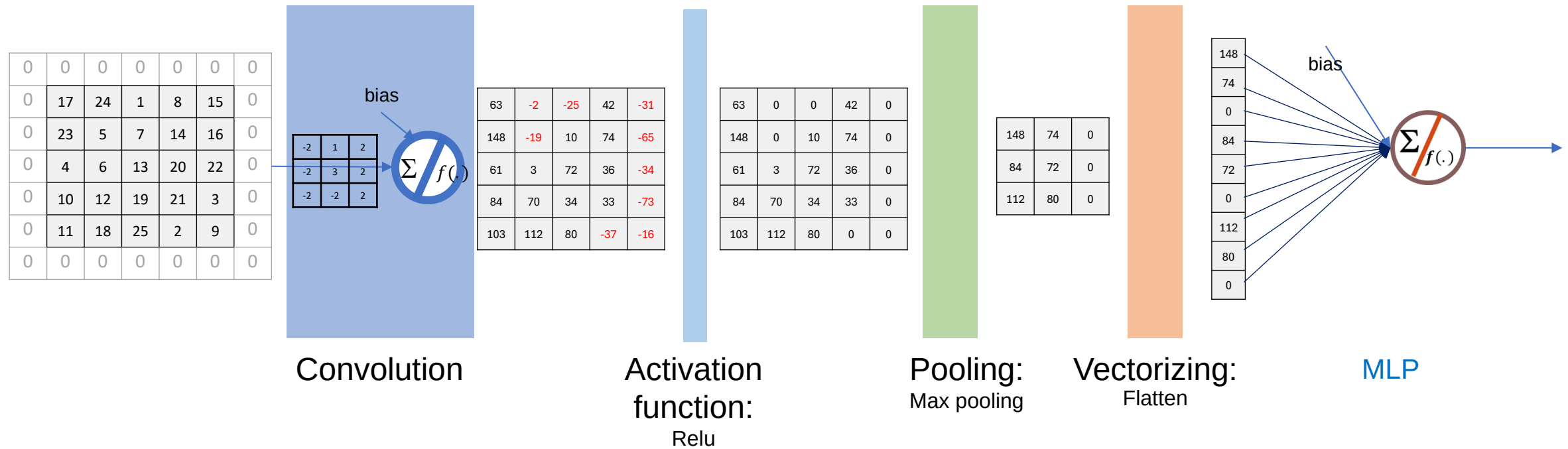


Vectorizing

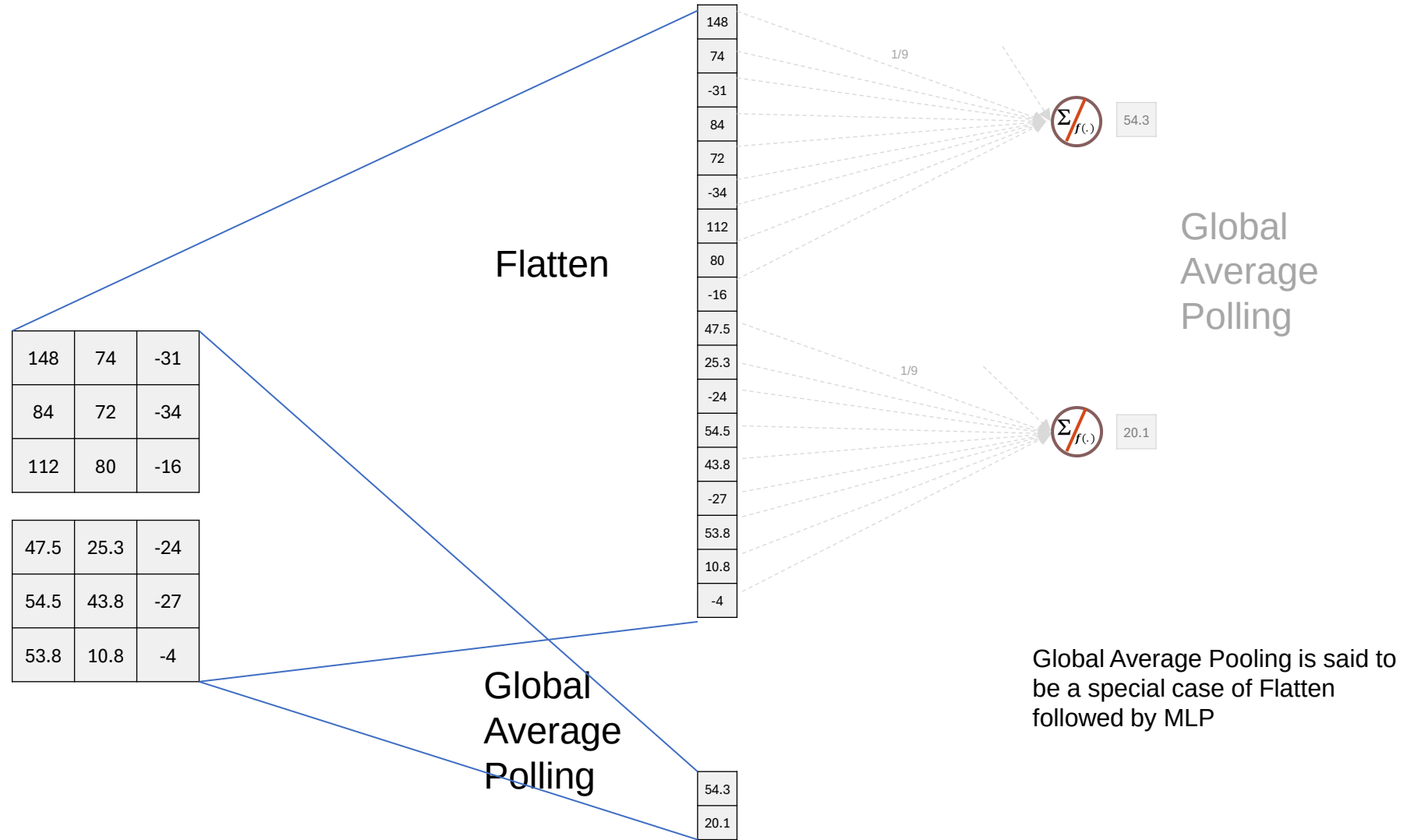
Converts feature maps to 1D vector



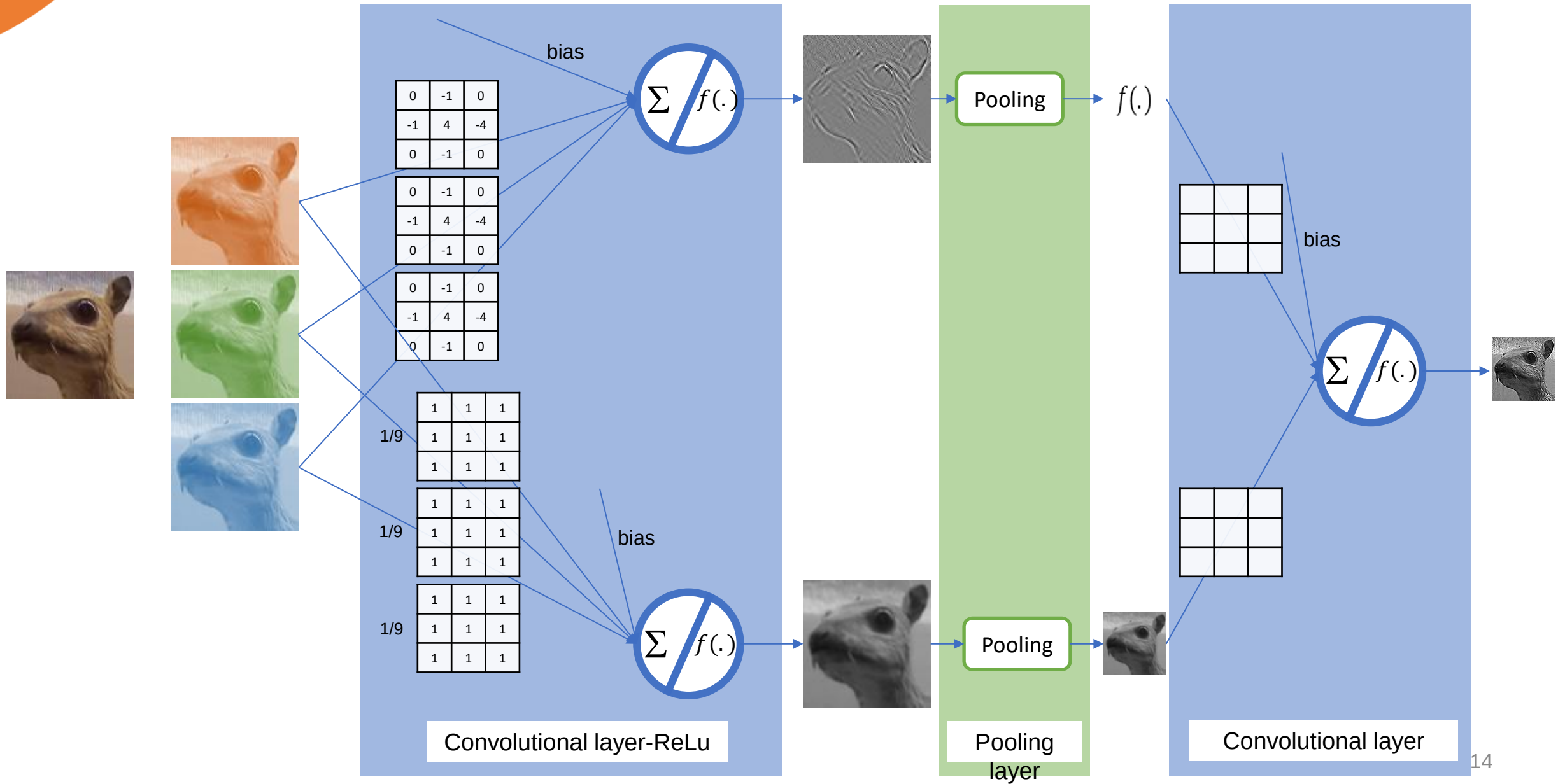
Convolutional Neural Network



Vectorizing: Global Average Pooling vs Flatten

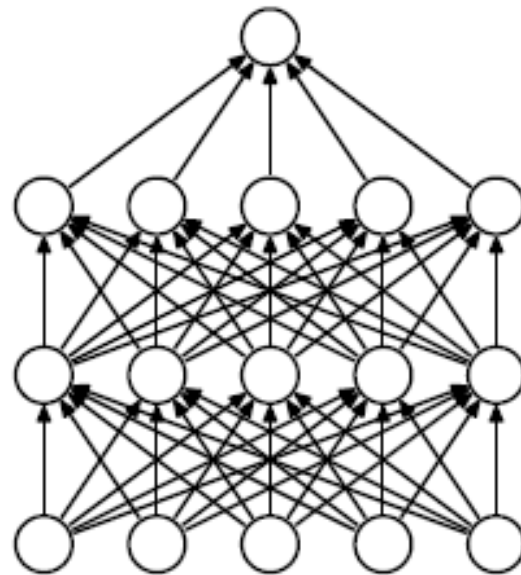


Convolutional Neural Network: multiple channels

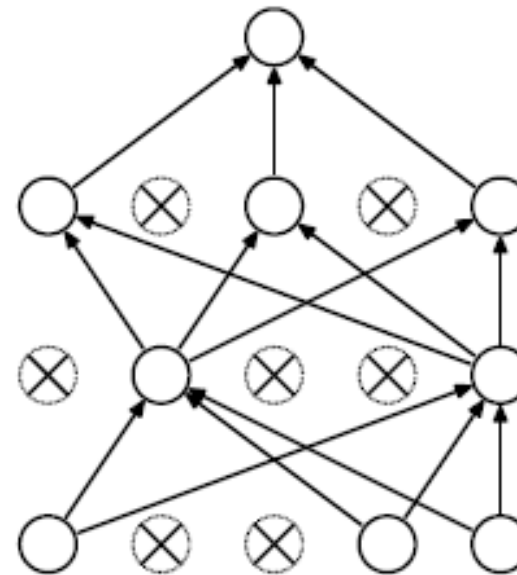


Generalization technique: Dropout layer

- The term “dropout” refers to dropping out units in a neural network.
- By dropping a unit out, we mean temporarily removing it from the network, along with all its incoming and outgoing connections.



(a) Standard Neural Net



(b) After applying dropout.

CNN Training

- Batch Gradient Descent
 - compute the gradient from all n training samples
- Stochastic Gradient Descent
 - compute the gradient from 1 training sample
- Mini-Batch Gradient Descent
 - compute the gradient from $1 < k < n$ training samples

Classification loss: Cross Entropy Loss

- We can use cross entropy to get a measure of dissimilarity between two probability distributions.
- Cross-entropy loss, or log loss, measure the performance of a classification model whose output is a probability value between 0 and 1.
- It is preferred for classification, while mean squared error (MSE) is one of the best choices for regression.
- The cross entropy of two probability distributions $p(x)$ and $q(x)$ is defined as $-\sum q(x) \log(p(x))$:

$$L = - \sum_{c \in \mathcal{C}} y_c \log(a_c)$$

- where y_c is the true class probability and a_c is the estimated class probability.

Classification loss: Cross Entropy Loss

- For **multiclass classification**, cross entropy known as **categorical cross entropy** is calculated as a sum of separate loss for each class label per observation. Mathematically, it is defined as the following equation:

true probability (class label)

y_1	y_2	y_3	y_4	y_5	y_6
0	1	0	0	0	0

predicted probability (model outputs)

a_1	a_2	a_3	a_4	a_5	a_6
0.1	0.8	0.1	0	0	0

$$L = - \sum_k y_k \log(a_k)$$

Not 0 but close to 0

where y_k is the true probability or the class label (0 or 1) and a_k denotes the predicted probability which is the output of output neuron k .

$$L = -(0 \log 0.1 + 1 \log 0.8 + 0 \log 0.1 + 0 \log 0 + 0 \log 0) \\ = 0.32192809$$

Classification output activation function: SoftMax

- The softmax function is defined as

$$f(z_k) = \frac{e^{z_k}}{\sum_c e^{z_c}}$$

- where the denominator $\sum_c e^{z_c}$ is the sum over all the classes.

$$a_k = f(z_k)$$

z_1	z_2	z_3	z_4	z_5	z_6
10	12	10	8	8	8



a_1	a_2	a_3	a_4	a_5	a_6
0.1	0.8	0.1	0	0	0

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

CNN Applications and output layer

Classification **Semantic Segmentation** **Object Detection** **Instance Segmentation**



CAT

No spatial extent



GRASS, CAT, TREE, SKY

No objects, just pixels



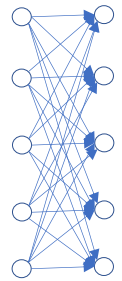
DOG, DOG, CAT

Multiple Object

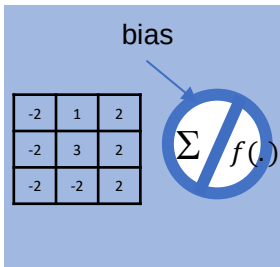


DOG, DOG, CAT

This image is CC0 public domain



Perceptron output

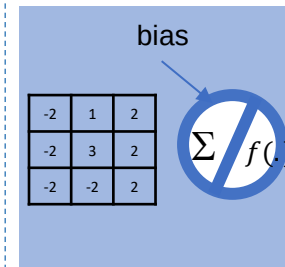


63	-2	-25	42	-31
148	-19	10	74	-65
61	3	72	36	-34
84	70	34	33	-73
103	112	80	-37	-16

(invert)
Convolutional output

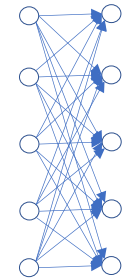


Perceptron output



63	-2	-25	42	-31
148	-19	10	74	-65
61	3	72	36	-34
84	70	34	33	-73
103	112	80	-37	-16

(invert)
Convolutional output



Perceptron output

Data, Data, Data

Machine learning == Data-driven intelligence

<https://www.image-net.org/>

1000 object classes

contains

- **1,281,167** training images,
- **50,000** validation images
- **100,000** test images

<https://paperswithcode.com/sota/image-classification-on-imagenet>



Some of Deep Learning Models
for Image Classification

ImageNet Classification Error (Top 5)

[227x227x3] INPUT

2012: AlexNet
5 conv. layers

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

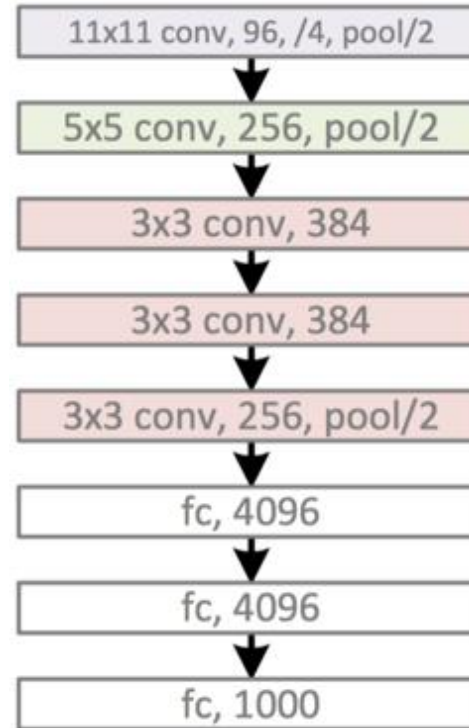
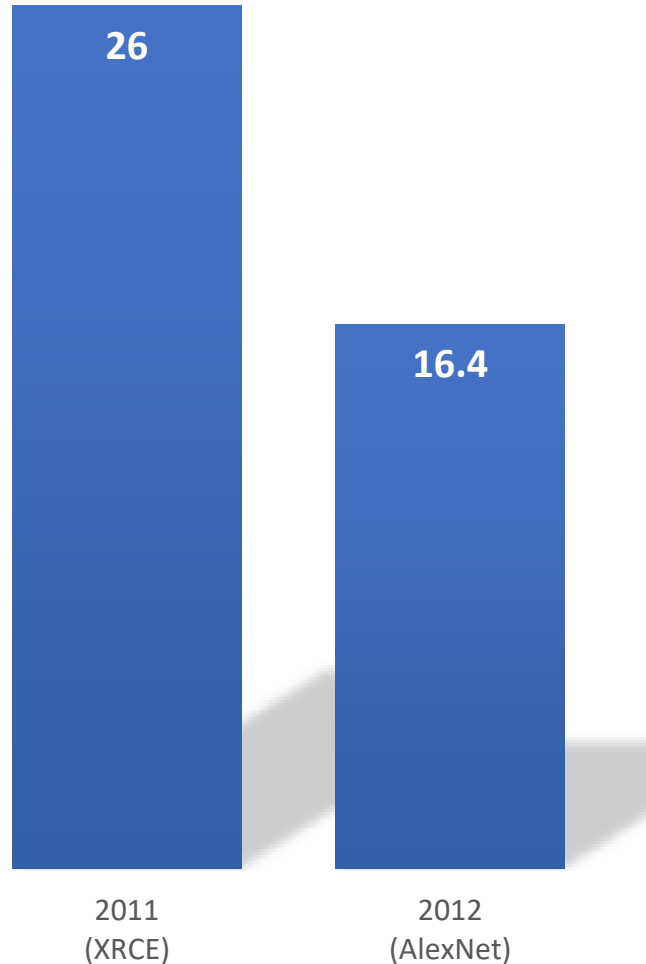
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

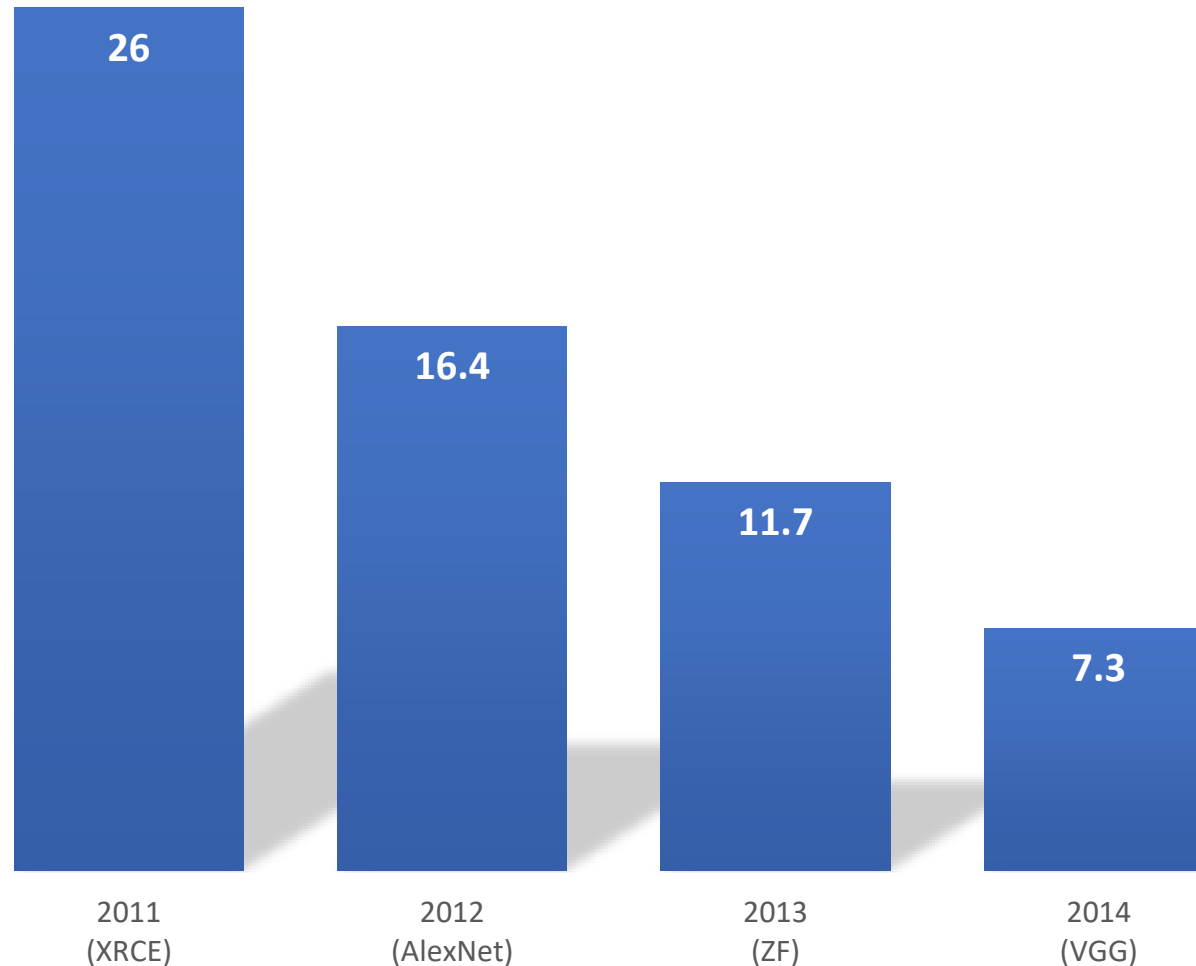
[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)



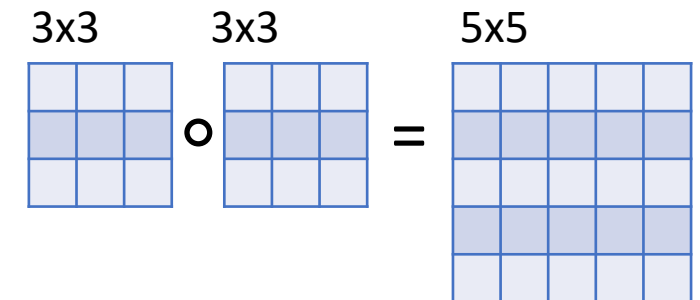
Error: 16.4%

ImageNet Classification Error (Top 5)



Error: 7.3%

2014: VGG 16/19 conv. layers
 Small convolutional kernels: 3x3
 ReLu non-linearities
 >100 million parameters.



25 coefficients, but only
 18 degrees of freedom

How Deep a Stack of Layers Can We Train

Try backpropagation to train deep NNs with larger number of layers:

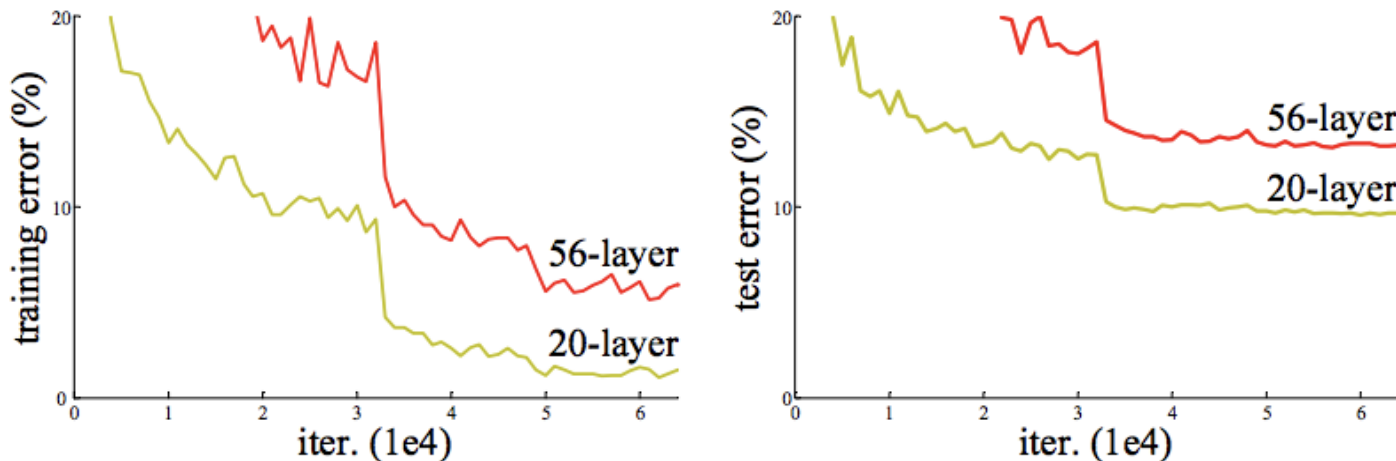
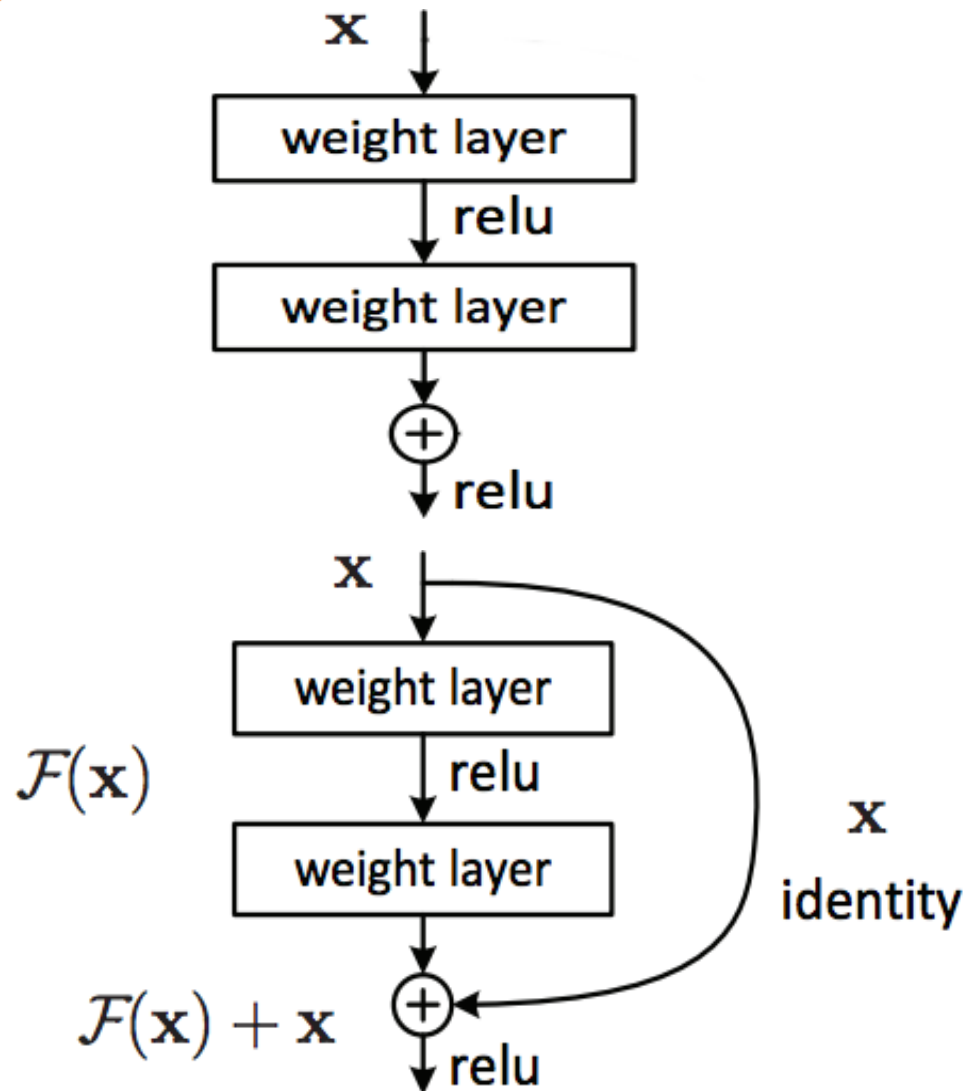


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

From [He, et.al., 2015](#)

- Possible explanations for these results:
 - Vanishing gradients?
 - Exploding gradients?
 - Overfitting?
- None of the above!
 - Vanishing gradients – batch normalization
 - Exploding gradients – gradient clipping
 - Overfitting – dropout regularization

How Deep a Stack of Layers Can We Train?



Ideally a subset of stacked layers learns a nonlinear function, $\mathcal{H}(x)$

- Alternatively, learn a **residual function**

$$F(x) = \mathcal{H}(x) - x$$

- Rearrange terms to find the **residual mapping** function
- Layers now have an **identity shortcut**

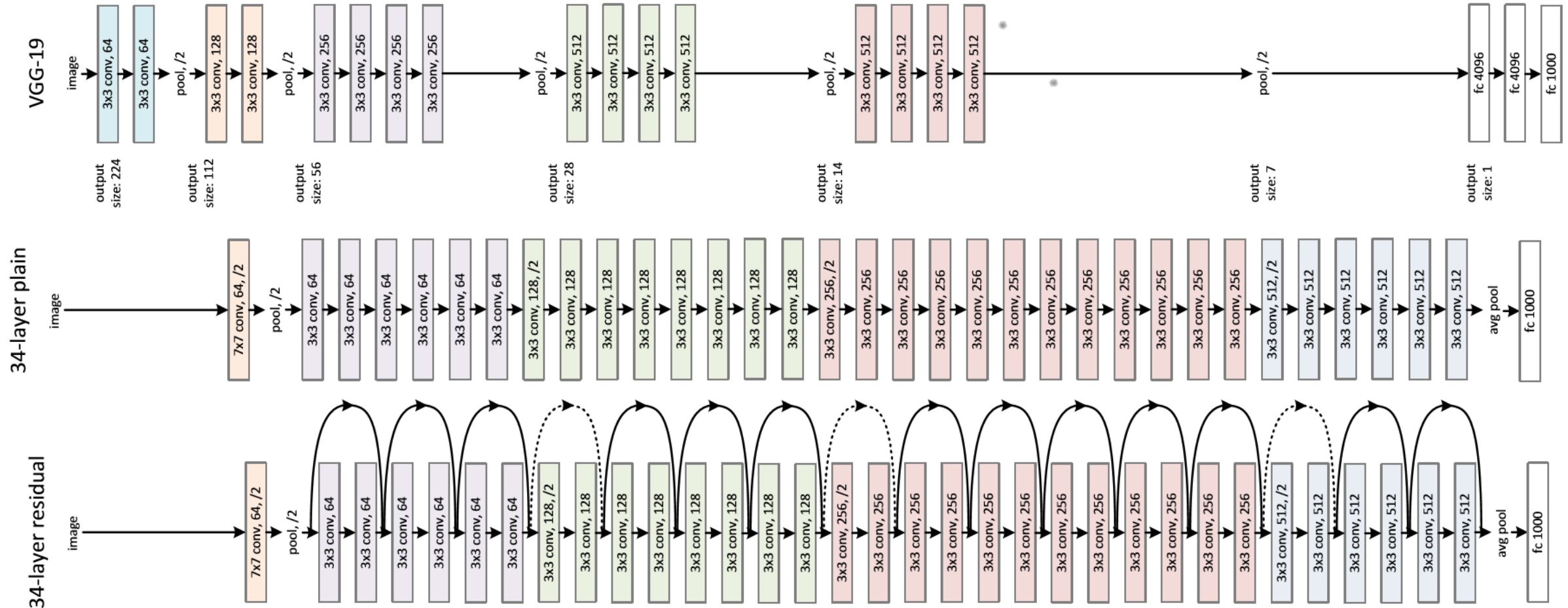
Figure 2. Residual learning: a building block.

From [He, et.al., 2015](#)

ResNet Representation with Residuals

- Start with VGG19 architecture
- 34 stacked layer network – poor training characteristics
- 34 layer ResNet with **identity shortcuts**

From [He, et.al., 2015](#)



ResNet Representation with Residuals

How well does ResNet work?

- 18 and 34 layers stacked network performance
- 18 and 34 layer ResNets

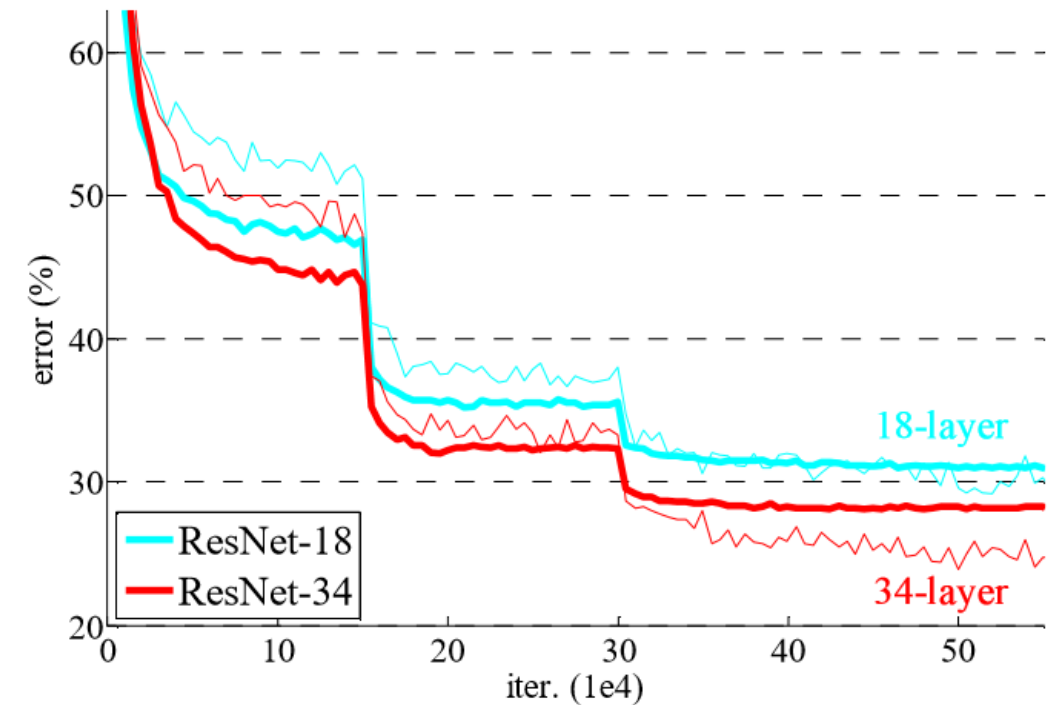
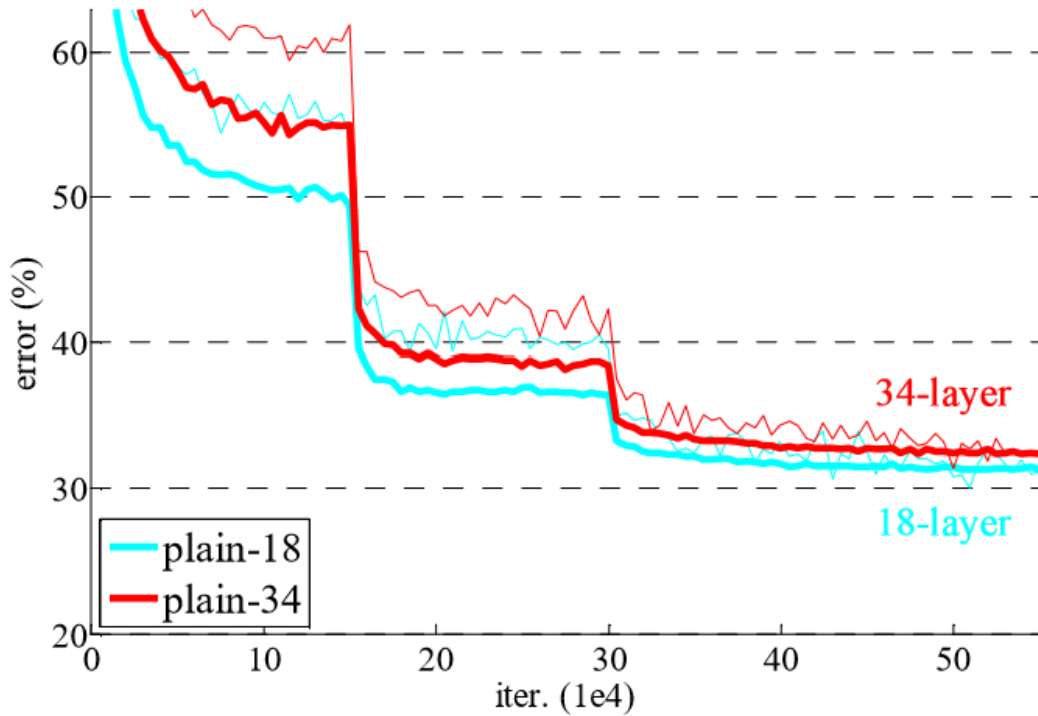
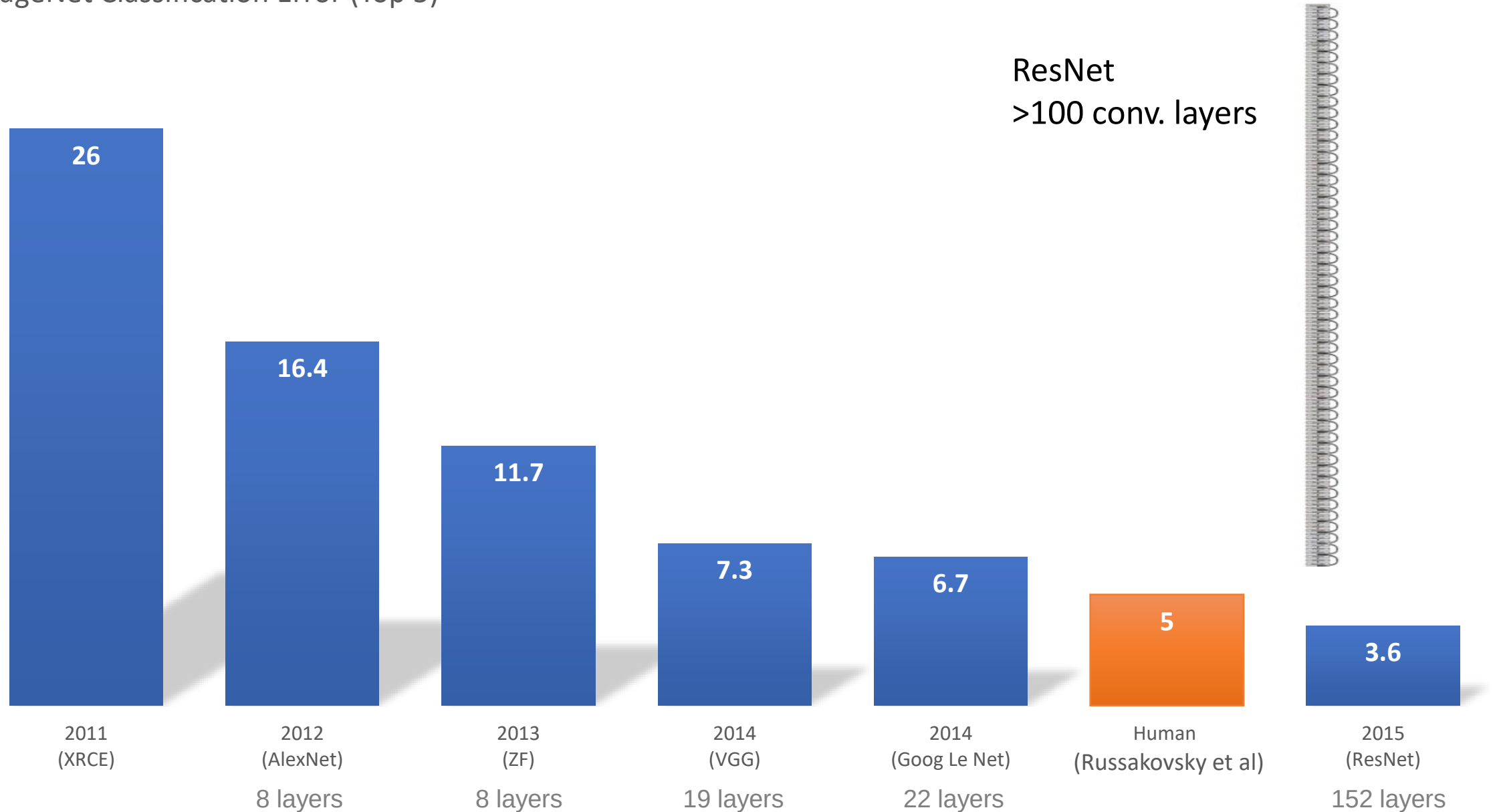


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

From [He, et.al., 2015](#)

ImageNet Classification Error (Top 5)

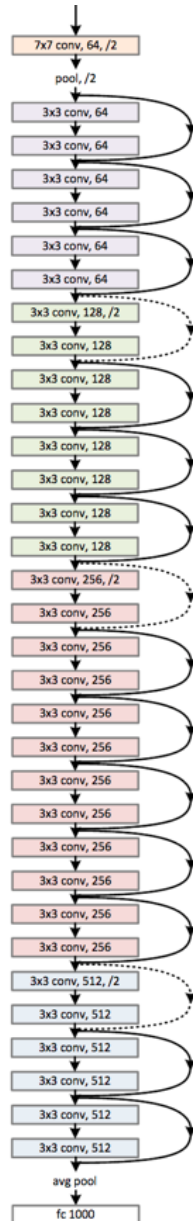
ResNet
>100 conv. layers



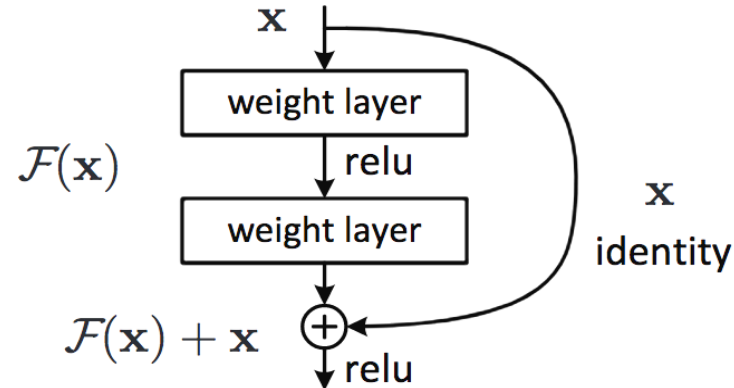


Residual blocks

ResNet >100 conv. layers



Residual Blocks



Main developments

- Increased depth possible through **residual blocks**

Why do they work?

- Gradients can propagate faster (via the identity mapping)
- Within each block, only small residuals have to be learned

Residual block in pytorch

```
# Define a residual block with skip connection
class ResidualBlock(nn.Module):
    def __init__(self, in_channels):
        out_channels = in_channels
        super(ResidualBlock, self).__init__()

        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(out_channels)

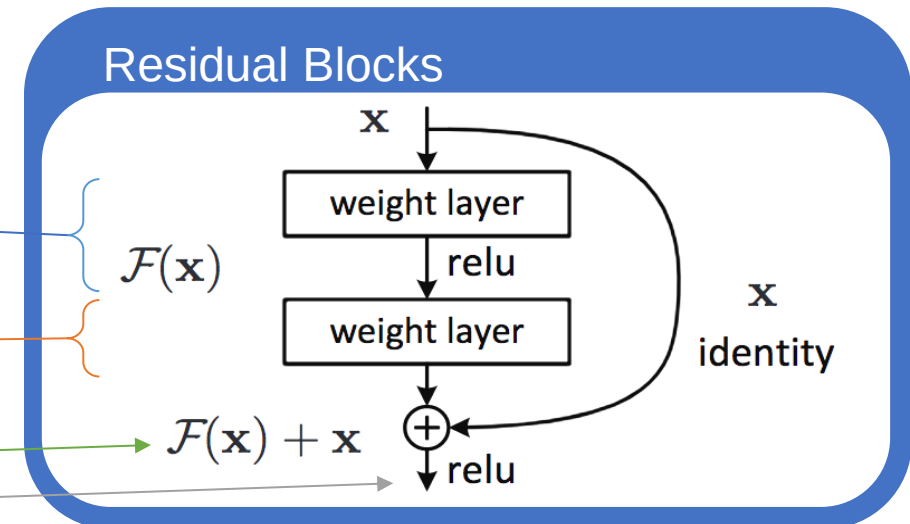
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        identity = x
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

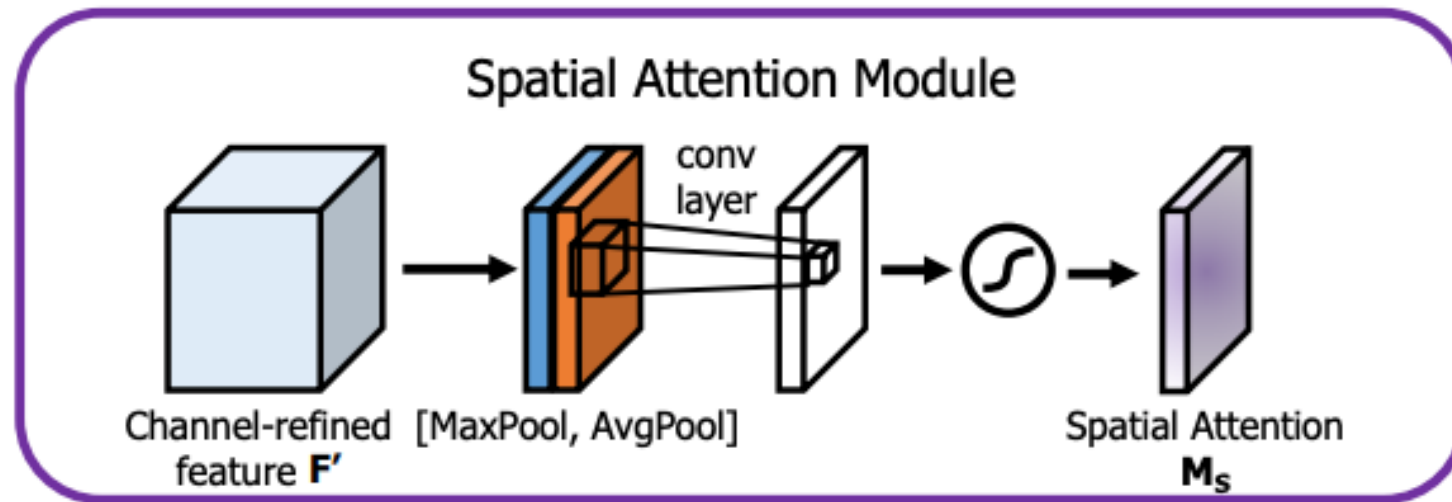
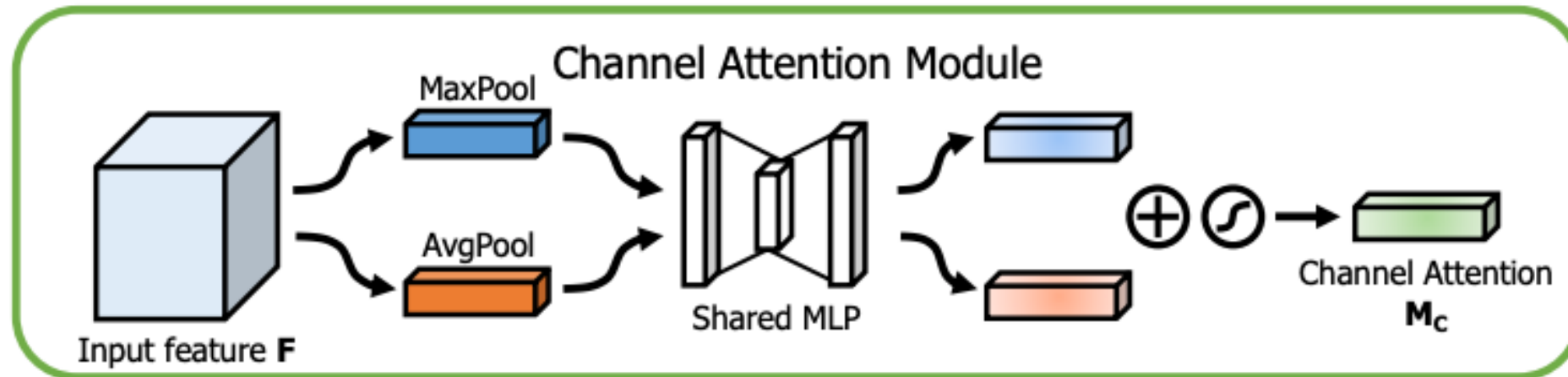
        out += identity  # Skip connection
        out = self.relu(out)
        return out
```





Convolutional Block Attention Module (CBAM)

Convolutional Block Attention Module(CBAM)

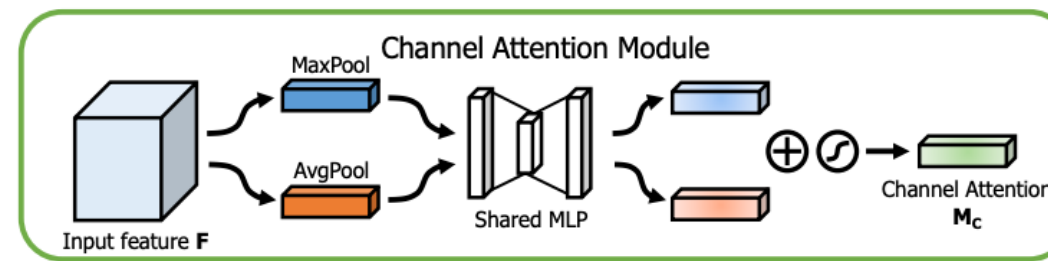
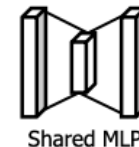


Channel Attention

```
class ChannelAttention(nn.Module):
    def __init__(self, in_channels, reduction_ratio=16):
        super(ChannelAttention, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)
        self.fc = nn.Sequential(
            nn.Conv2d(in_channels, in_channels // reduction_ratio, kernel_size=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(in_channels // reduction_ratio, in_channels, kernel_size=1)
        )
        # self.fc = nn.Sequential(
        #     nn.Linear(in_channels, in_channels // reduction_ratio),
        #     nn.ReLU(inplace=True),
        #     nn.Linear(in_channels // reduction_ratio, in_channels)
        # )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = self.fc(self.avg_pool(x))
        max_out = self.fc(self.max_pool(x))
        out = avg_out + max_out
        return self.sigmoid(out)
```

reduce
parameter
overhead

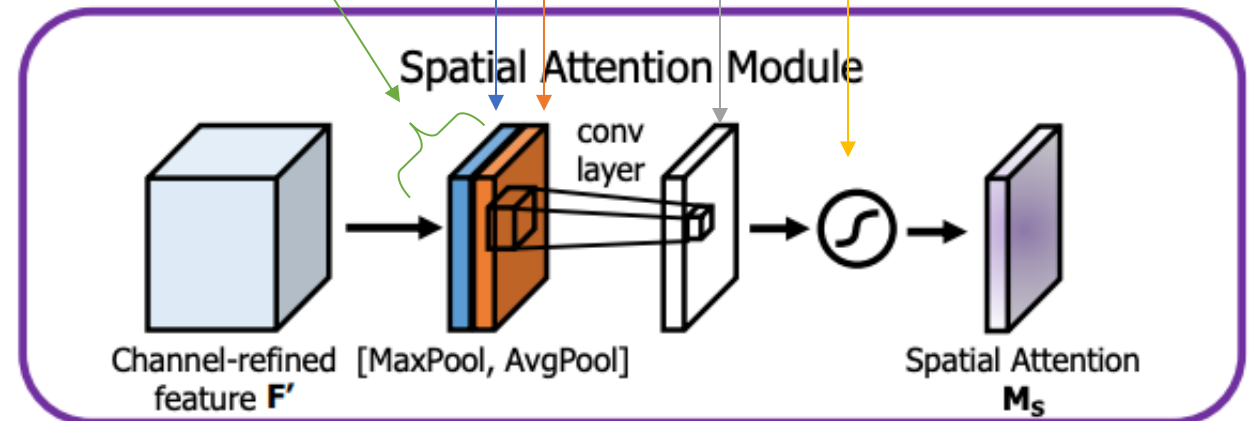


Spatial Attention

```
# Define the Spatial Attention Module
class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super(SpatialAttention, self).__init__()

        self.conv = nn.Conv2d(2, 1, kernel_size=kernel_size, padding=kernel_size//2)
        self.sigmoid = nn.Sigmoid()

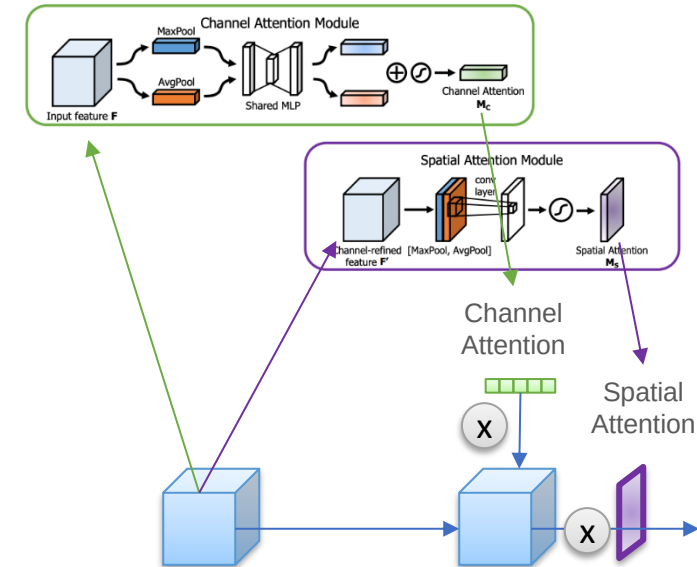
    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        x = torch.cat([avg_out, max_out], dim=1)
        x = self.conv(x)
        return self.sigmoid(x)
```



Convolutional Block Attention Module (CBAM)

Define the CBAM-like Attention Module

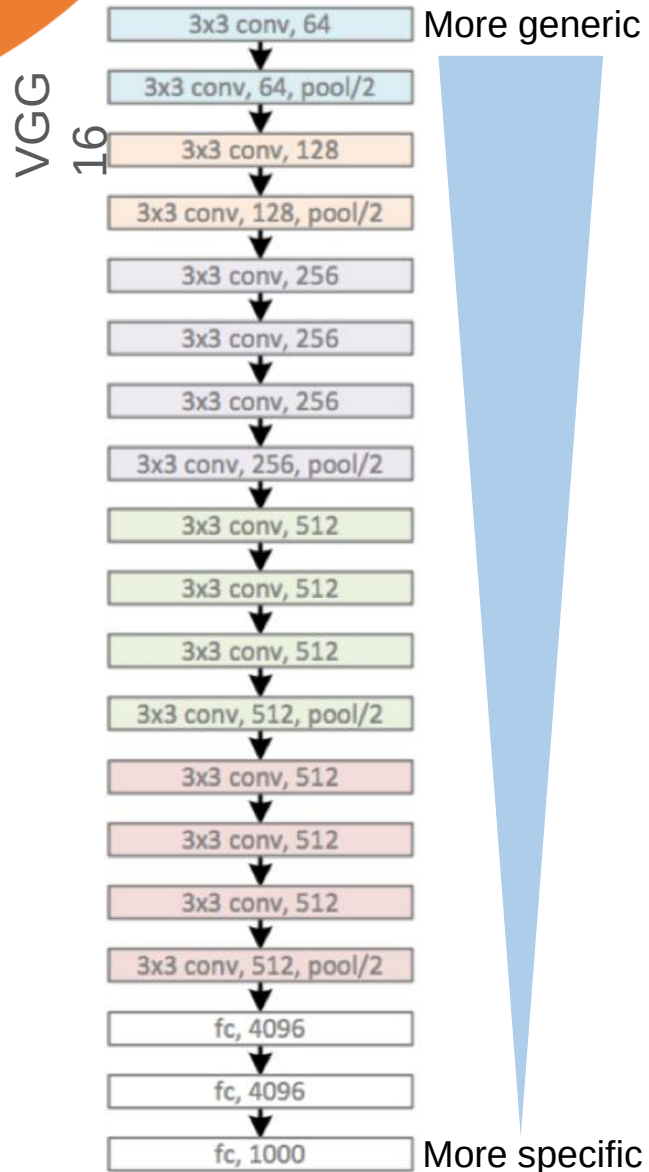
```
class CBAMAttention(nn.Module):  
    def __init__(self, in_channels):  
        super(CBAMAttention, self).__init__()  
  
        self.channel_att = ChannelAttention(in_channels)  
        self.spatial_att = SpatialAttention()  
  
    def forward(self, x):  
        x = x * self.channel_att(x)  
        x = x * self.spatial_att(x)  
        return x
```



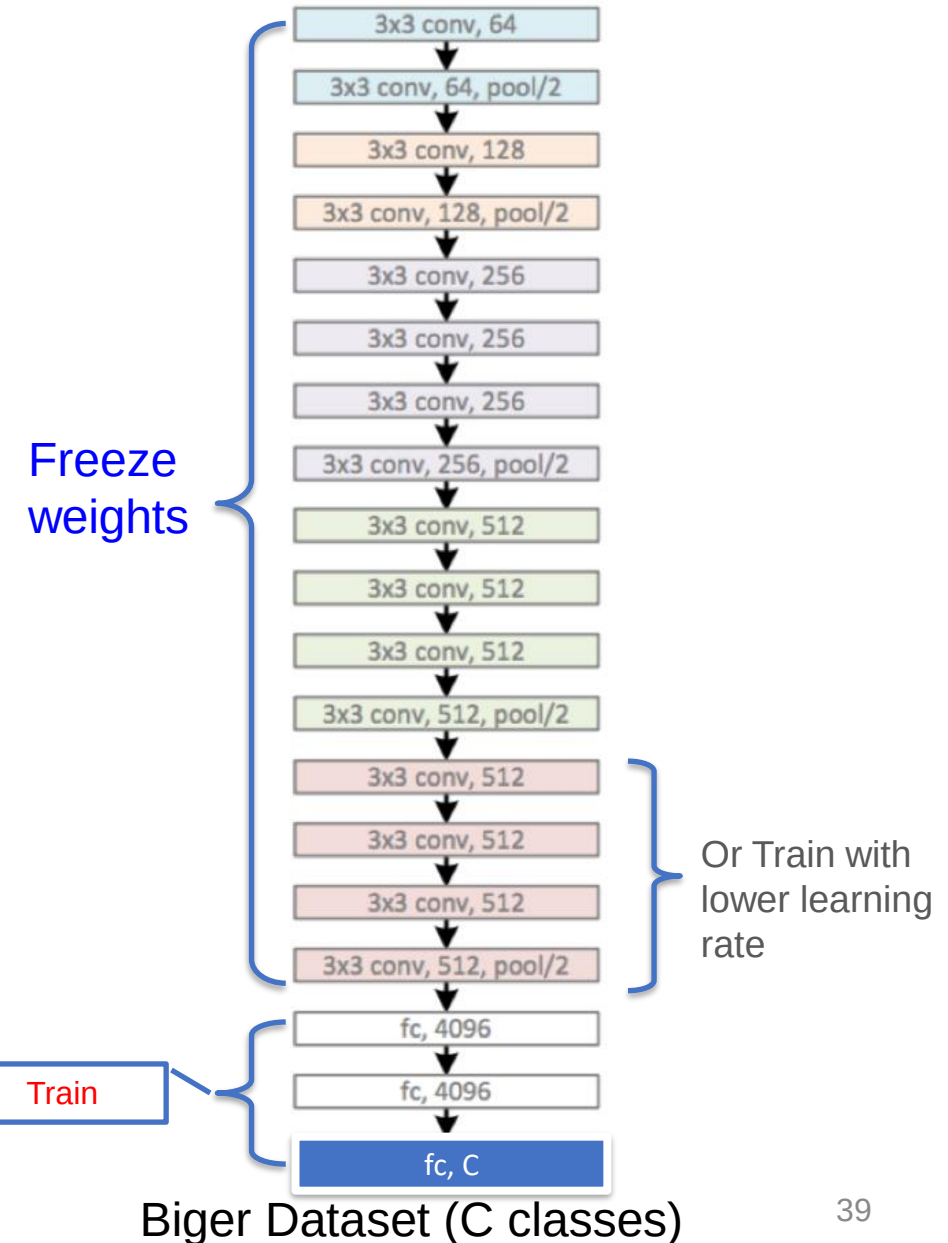
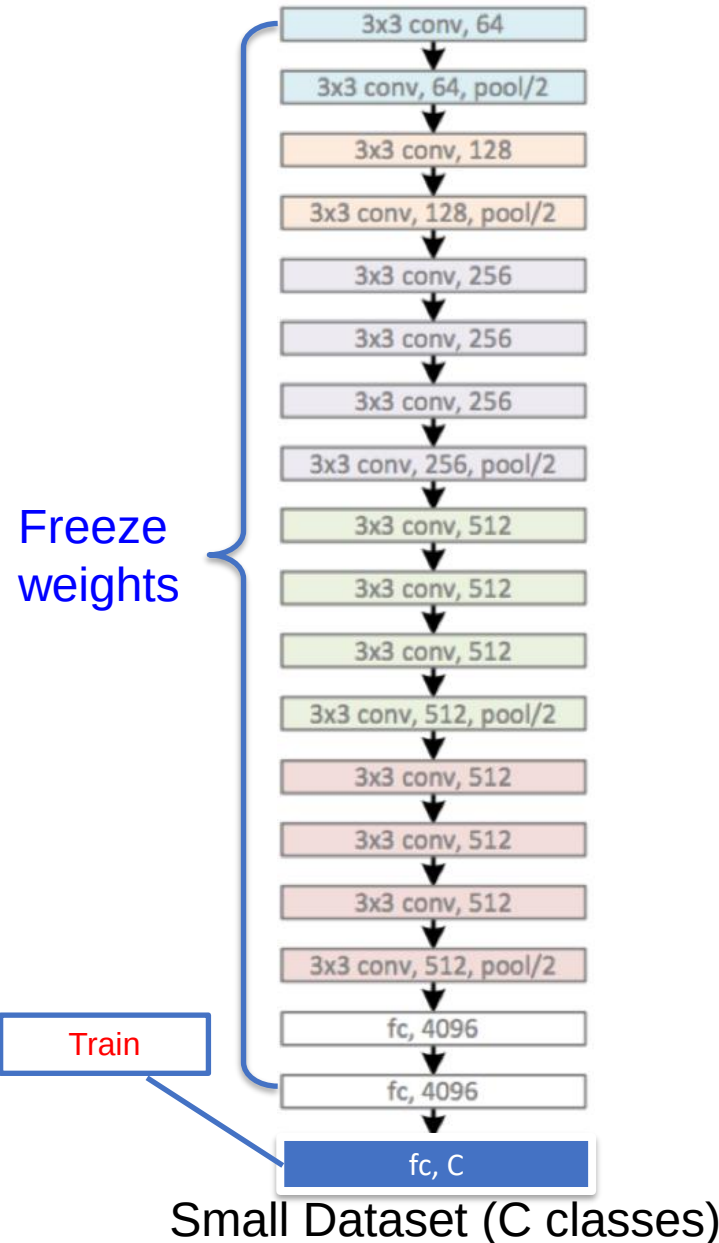


Transferred Learning

Transferred learning



Train on Imagenet



Popular Architectures

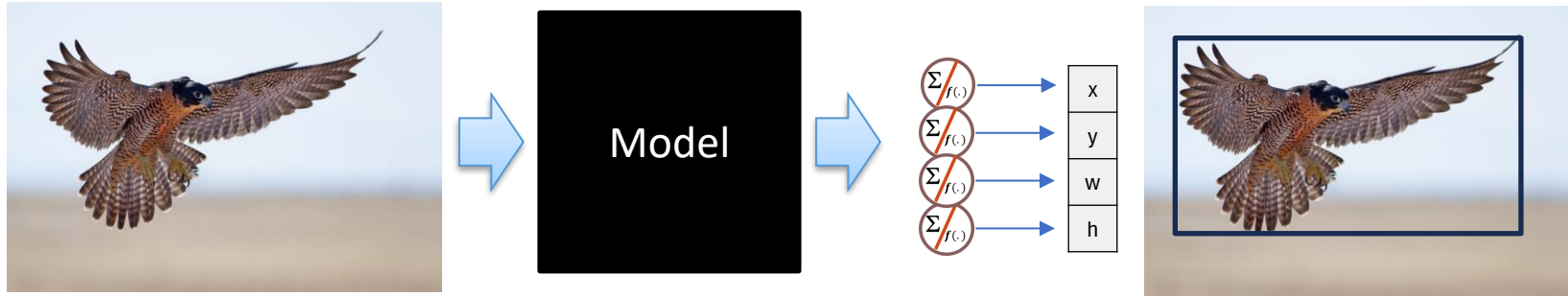
- LeNet-5: Early CNN designed for digit recognition.
- AlexNet: Breakthrough CNN model that won the ImageNet competition in 2012.
- VGGNet: Deeper architecture with small convolutional filters.
- GoogLeNet (Inception): Uses inception modules for efficient computation.
- ResNet: Introduces residual learning to enable very deep networks.
- EfficientNet: Optimized for parameter efficiency and accuracy.
- Transformers (ViTs): Vision Transformers leverage self-attention mechanisms for image processing.



(OPTIONAL)

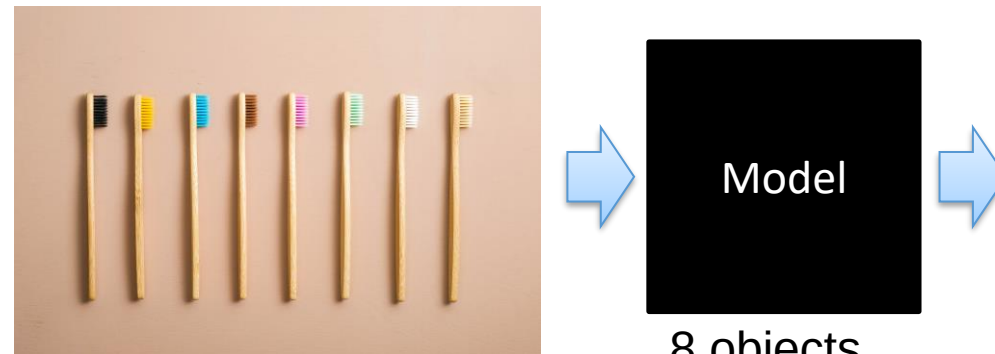
Object detection

Single object per image, Single class



1 object,
1 class model

Multiple objects per image, Single class

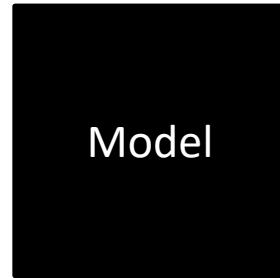


8 objects,
1 class model



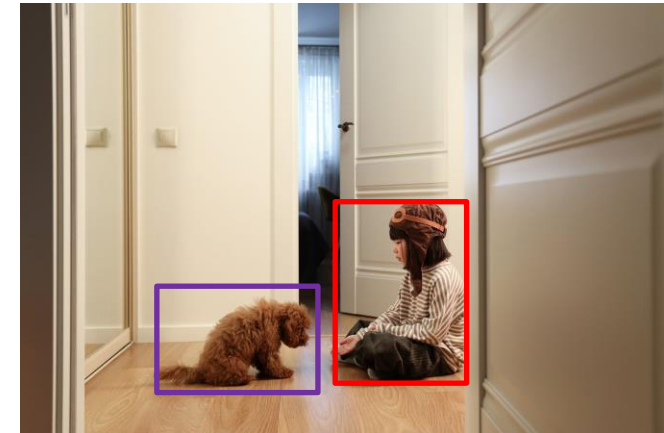
Object detection

Multiple objects per image, Multiple classes



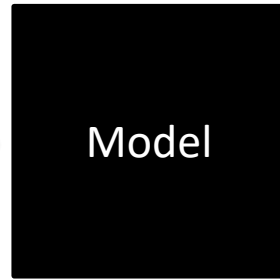
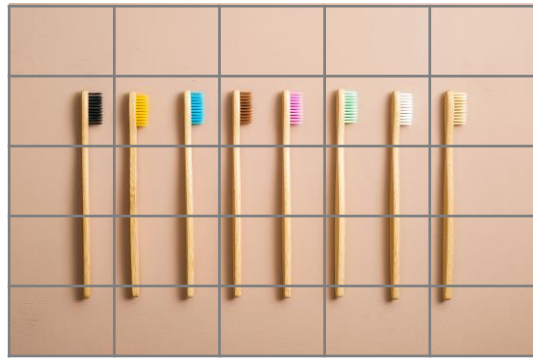
3 objects,
3 classes model

P1
P2
p3
x
y
w
h
P1
P2
p3
x
y
w
h
P1
P2
p3
x
y
w
h

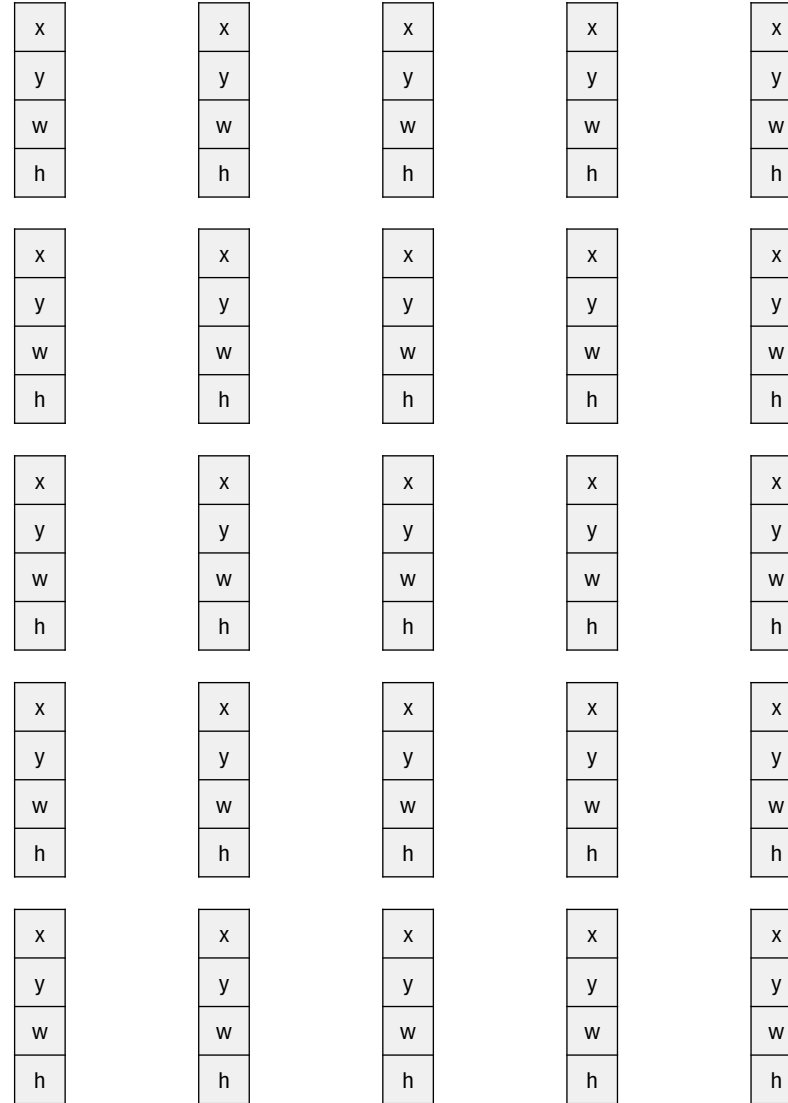


Object detection

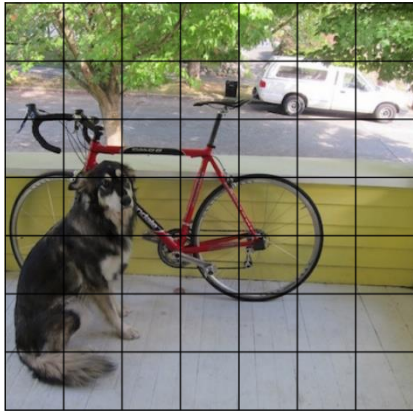
Multiple objects per image, Single class



1 object,
1 class model
With Grid search

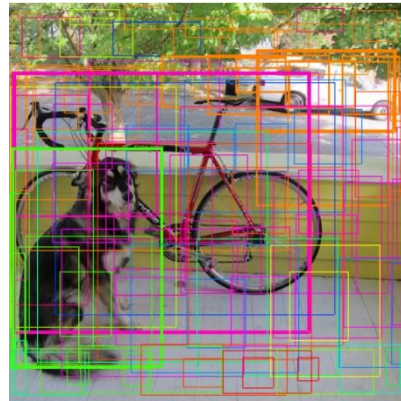


Object detection



YOLO

5 objects,
80 classes model
With 8x8 Grid search



Post
Processing

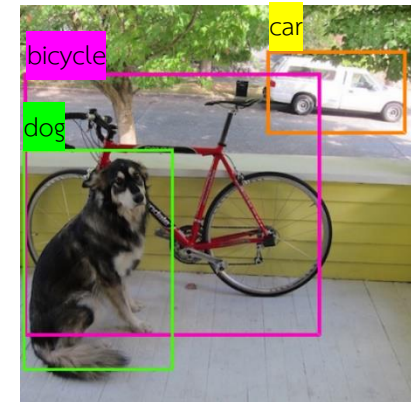


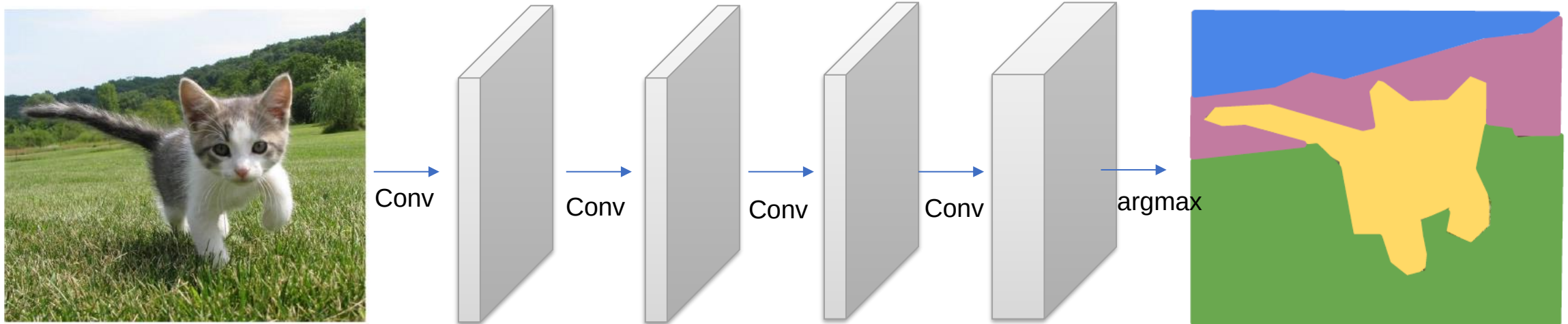
Image Segmentation



Semantic Segmentation

- Semantic Segmentation is a pixel-level image classification task where each pixel is assigned to a specific class.
- Each pixel gets labeled as belonging to a class

Idea



Problems:

- High-resolution images require more computations
- Memory usage increases quadratically with image size
- Running convolutions on full resolution is computationally expensive

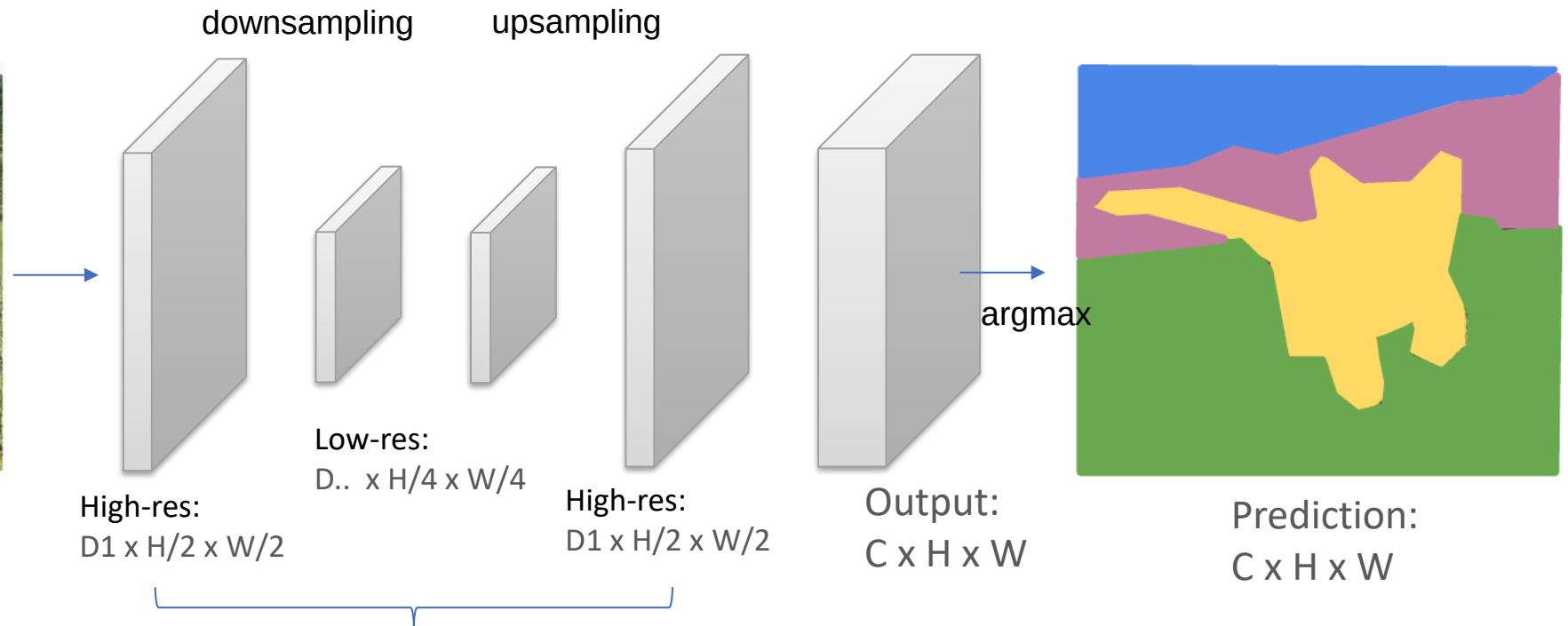
Semantic Segmentation

- Semantic Segmentation is a pixel-level image classification task where each pixel is assigned to a specific class.
- Each pixel gets labeled as belonging to a class

Idea



Input:
 $3 \times H \times W$



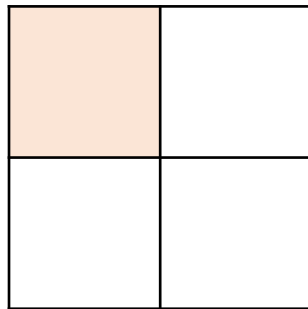
Autoencoder Model, Input=Output

Solution:

- Downsampling - Upsampling Strategy

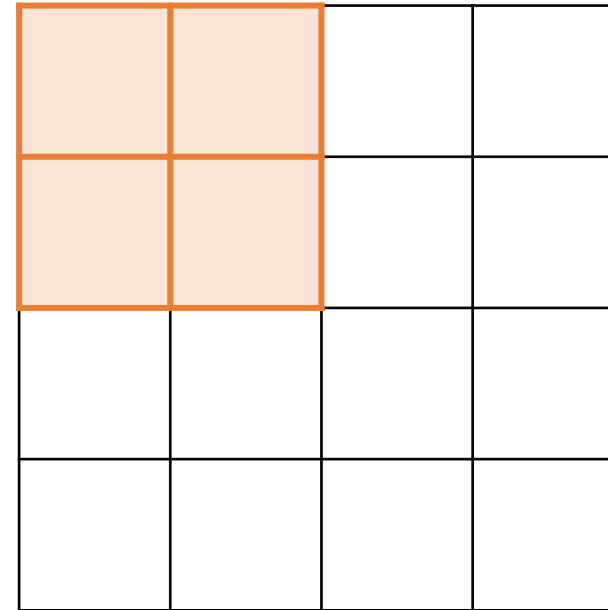
Learnable Upsampling: Transposed Convolution

2 x 2 transposed convolution, stride 2



Input: 2 x 2

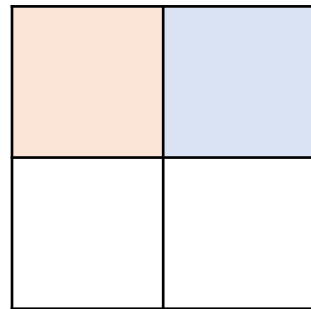
Input gives
weight for
filter



Output: 4 x 4

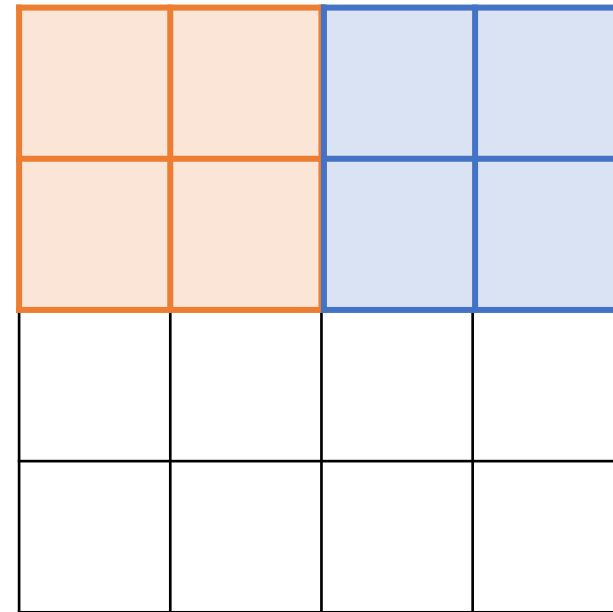
Learnable Upsampling: Transposed Convolution

2 x 2 transposed convolution, stride 2



Input: 2 x 2

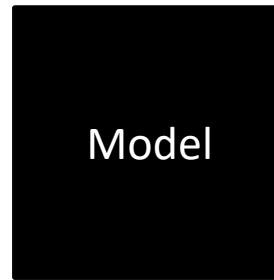
Input gives
weight for
filter



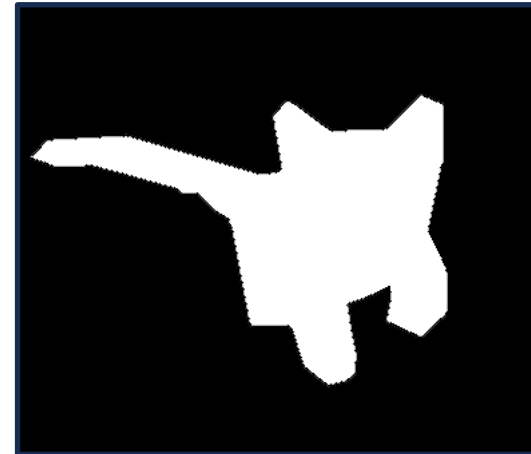
Output: 4 x 4

Filter moves 2 pixels in the
output for every one pixel
in the input

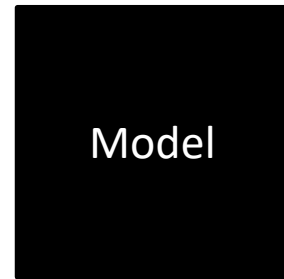
Semantic Segmentation



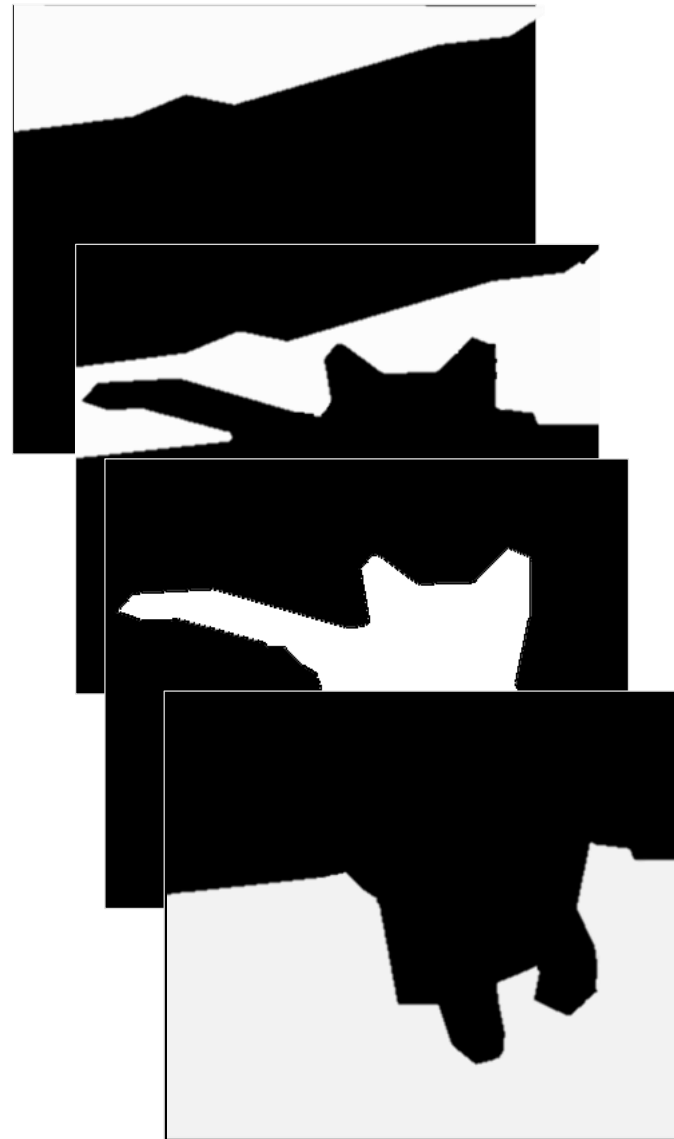
1 class



Semantic Segmentation

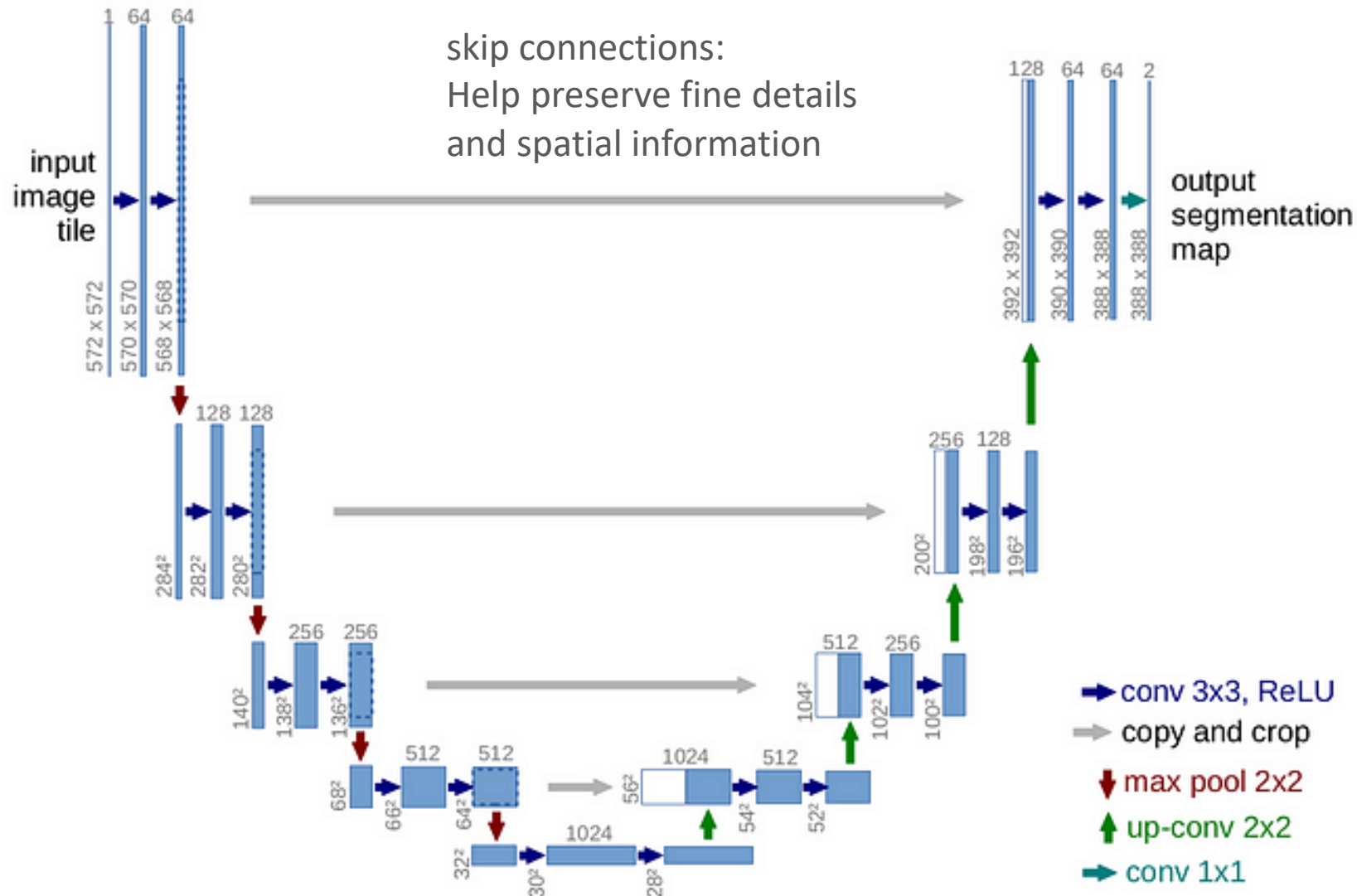


1 class



U-Net

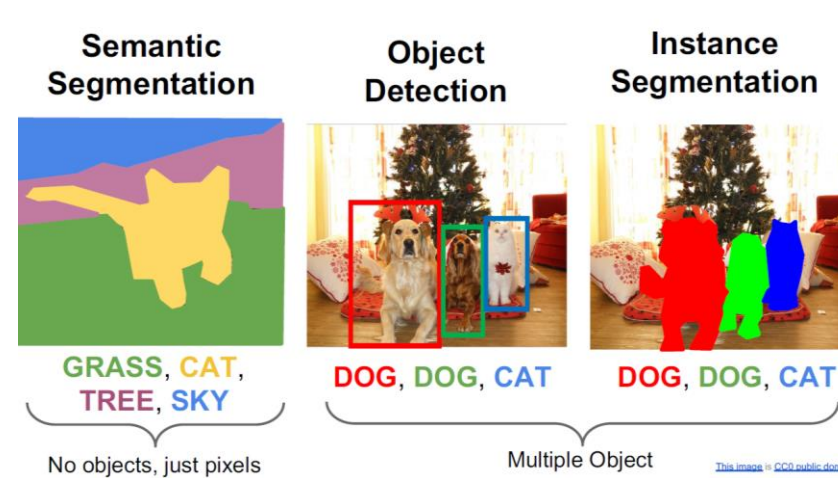
- U-Net is a popular convolutional neural network architecture primarily used for image segmer



Advanced models for image segmentation

Region Proposal Network

- R-CNN
- Fast R-CNN
- Faster R-CNN
- Mask R-CNN



Single-Stage Object Detectors

- [YOLO](#)
- SSD
- RetinaNet

Applications of Deep Learning in Computer Vision

- **Image Classification:**
 - Identifying the primary object in an image.
 - Widely used in tagging, content filtering, identifying diseases in medical images.
- **Object Detection:** Identifying and localizing objects in images (e.g., self-driving cars, detect pedestrians, traffic signs, surveillance security systems).
 - Two-Stage Detectors (R-CNN, Fast R-CNN, Faster R-CNN)
 - Single-Stage Detectors (YOLO, SSD, RetinaNet)
- **Semantic Segmentation:** Pixel-wise classification of images (e.g., medical imaging, scene understanding).
 - Semantic Segmentation: Assigning a class label to each pixel (FCN, U-Net, SegNet).
 - Instance Segmentation: Distinguishing individual objects of the same class (Mask R-CNN).
- **Face Recognition and Keypoint Detection:** Used in authentication, security, and social media.
- **Image/Video Captioning:** Combining vision and language models (CNN + RNN/Transformer)
- **Generative Models:** GANs and VAEs for image synthesis, super-resolution, and style transfer.

Current Trends and Future Directions

- **Self-Supervised Learning:** Reducing dependency on labeled data.
 - Leverage unlabeled data with contrastive learning (SimCLR, MoCo) or masked image modeling (MAE).
- **Vision and Language Integration:** Combining vision with other data modalities (text, audio, etc.).
 - Multimodal models (CLIP, DALL·E) enabling image-text tasks.
- **Edge AI and Model Compression:** Developing lightweight models for edge devices.
 - Pruning, quantization, knowledge distillation.
 - Deploying vision models on mobile devices, drones, IoT.
- **3D Computer Vision**
 - Depth estimation, 3D object detection, point cloud processing, AR/VR.
- **Neural Architecture Search (NAS)**
 - Automating design of optimal network architectures for given tasks.
- **Ethics, Regulation: Federated Learning**
 - Growing interest in regulating AI technologies, ensuring transparency and accountability.
 - Enabling privacy-preserving training across multiple devices.



PyTorch: Computational graph

Computational graph

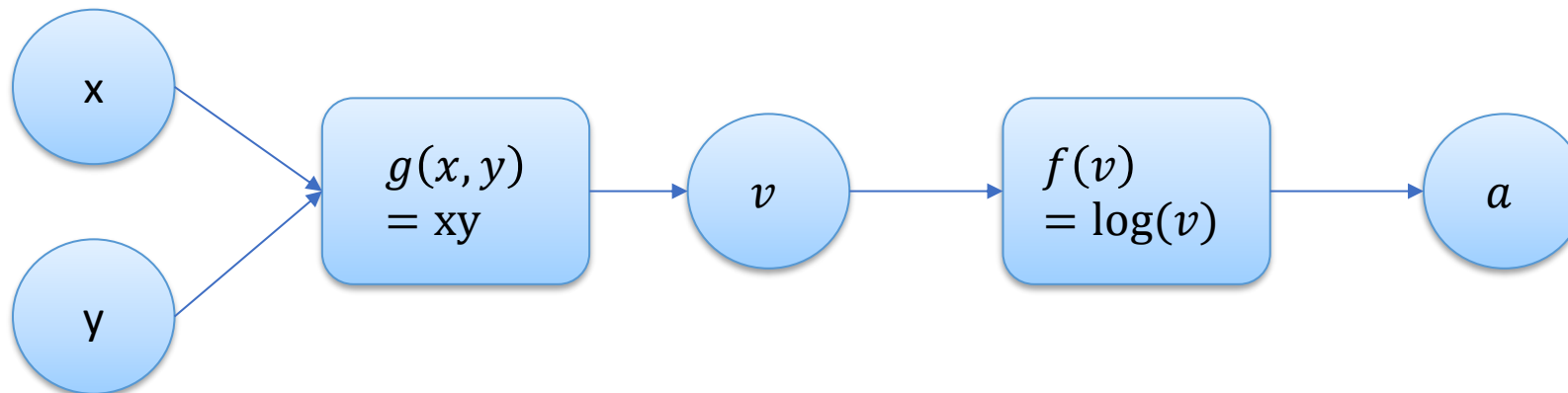
Given a composite function $f(g(x))$ we can calculate its derivative as $\frac{\partial}{\partial x} f(g(x)) = f'(g(x))g'(x)$

$$f(x, y) = \log(x * y)$$

- $f(x, y) = f(g()) = \log(g(x, y))$ where $g(x, y) = xy$

- $g(x, y) = xy = v$
- $f(x, y) = f(v) = a$

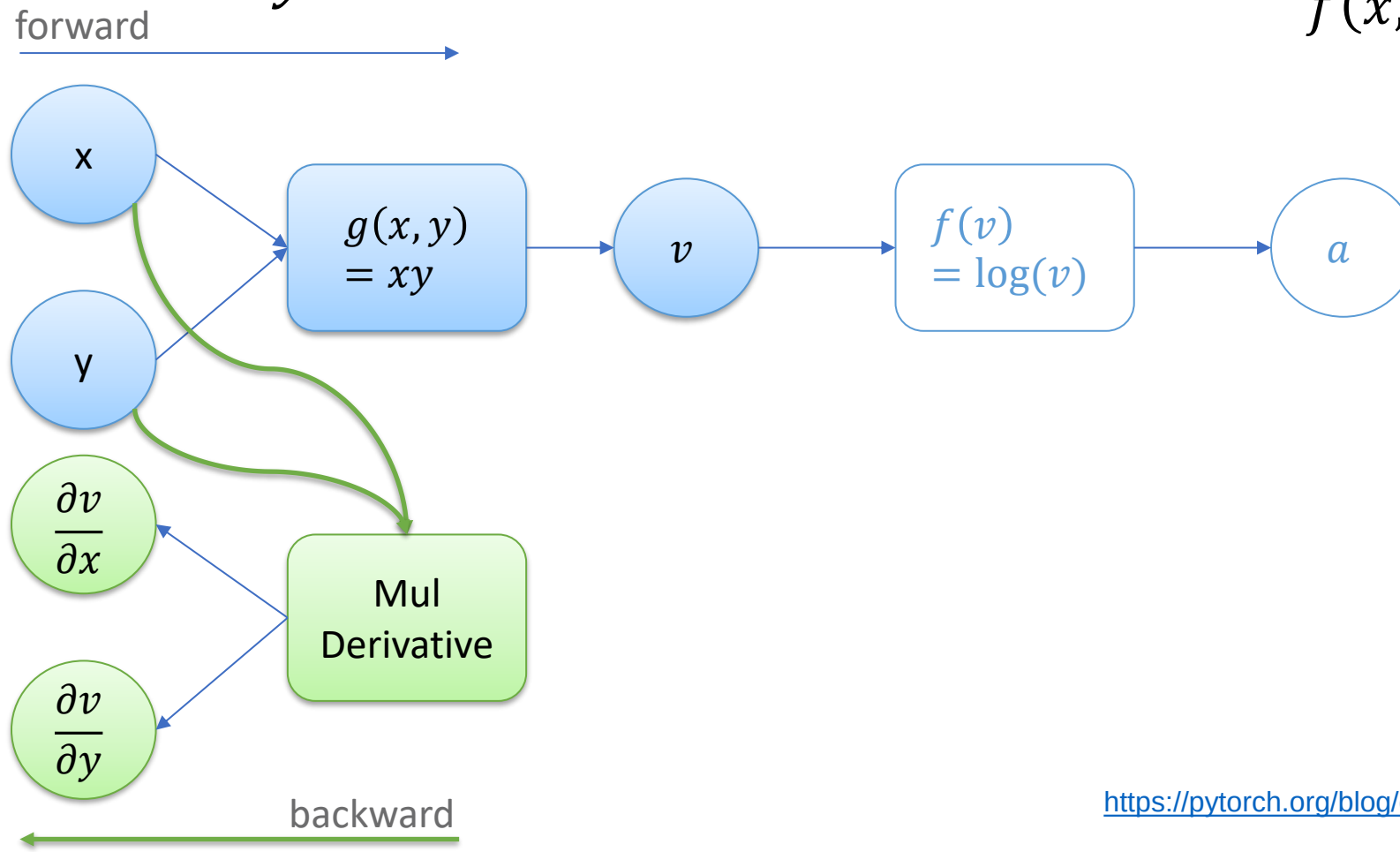
Computational graph



Computational graph: forward and computational graph extended

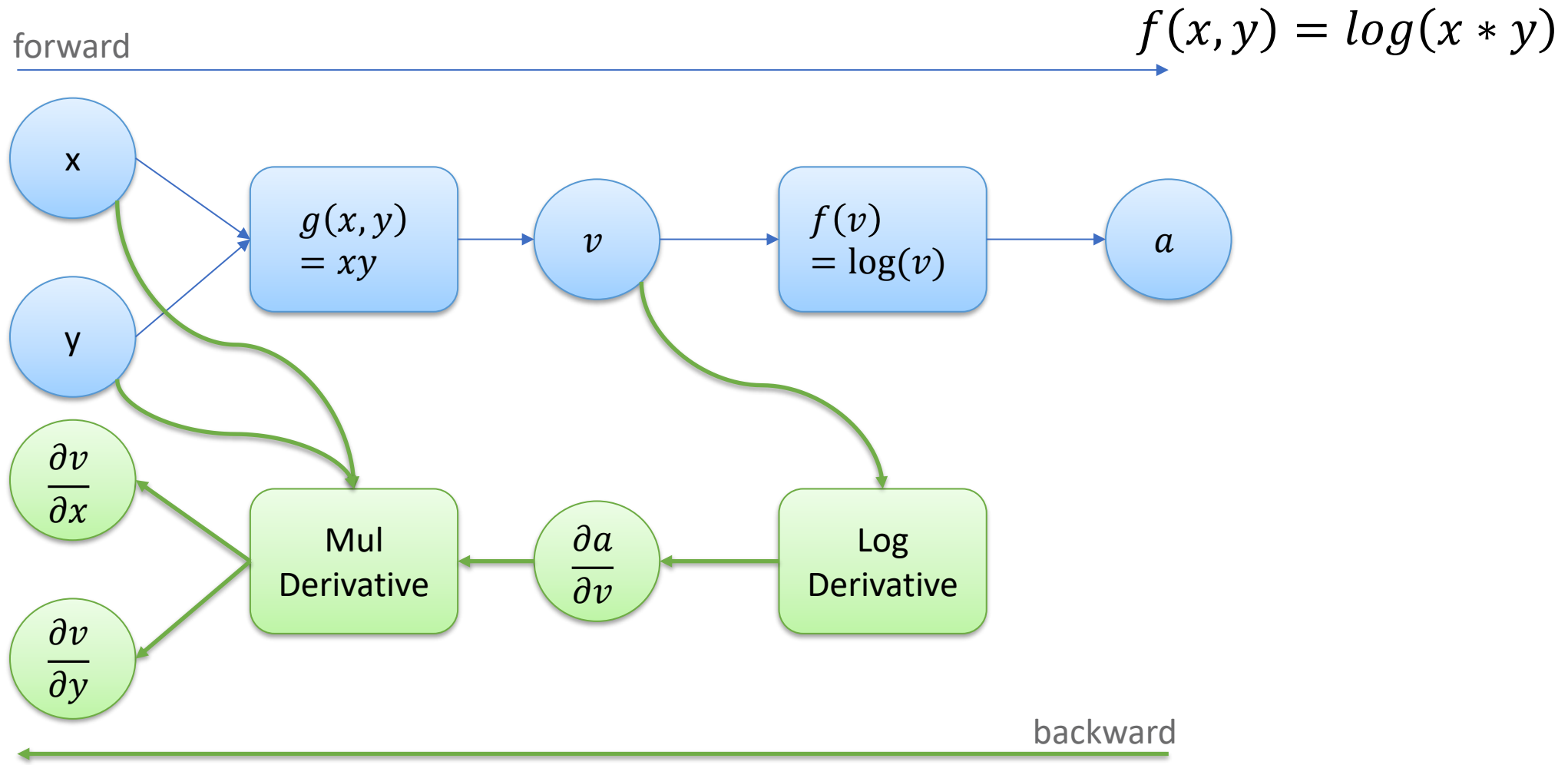
- $g(x, y) = xy = v$
- $\frac{\partial}{\partial x} g(x, y) = y, \frac{\partial}{\partial y} g(x, y) = x$

$$f(x, y) = \log(x * y)$$



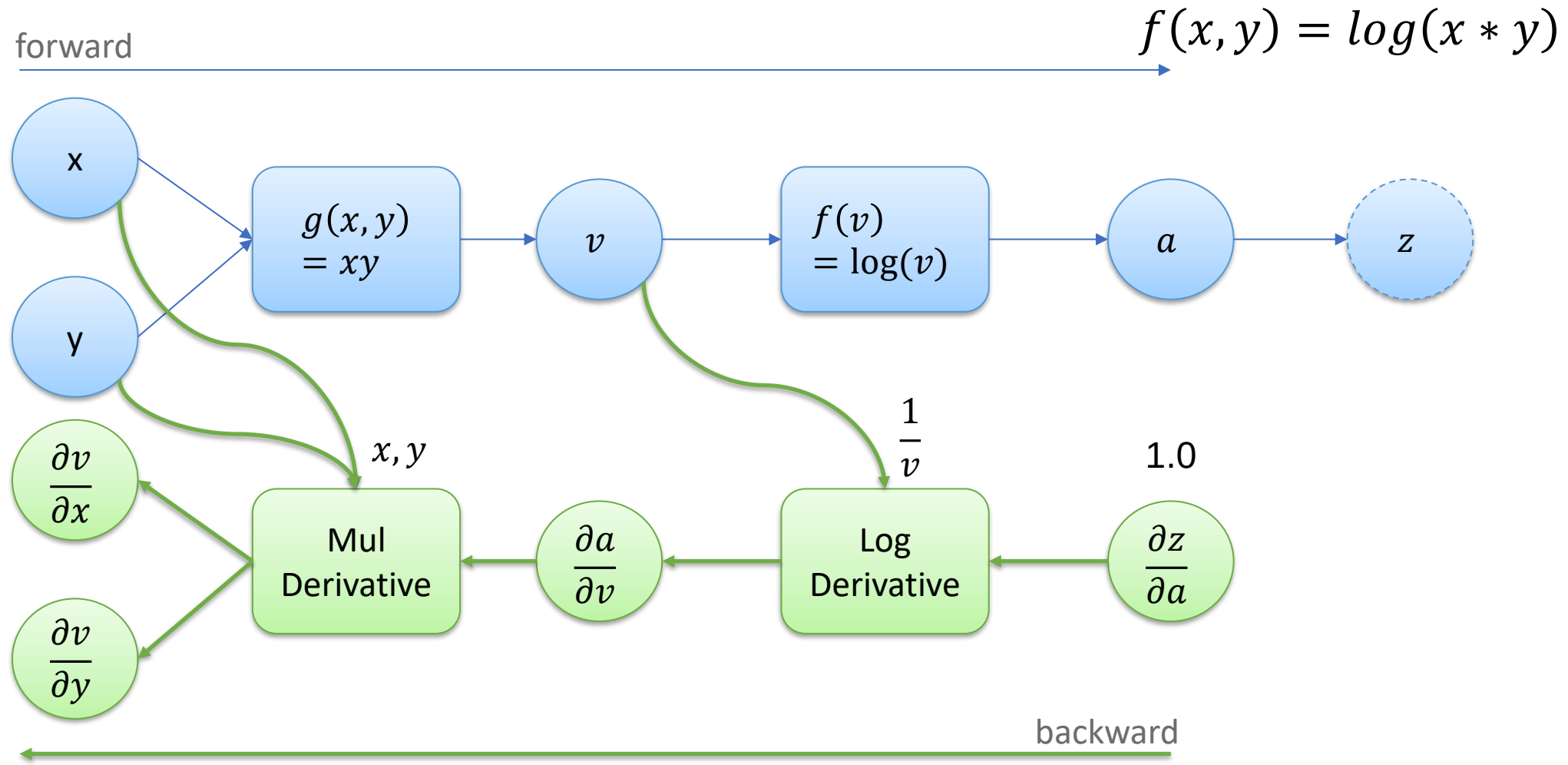
Computational graph: forward and computational graph extended

- $f(x, y) = f(v) = \log(v) = a$
- $\frac{\partial}{\partial v} f(v) = \frac{\partial a}{\partial v} = \frac{1}{v}$



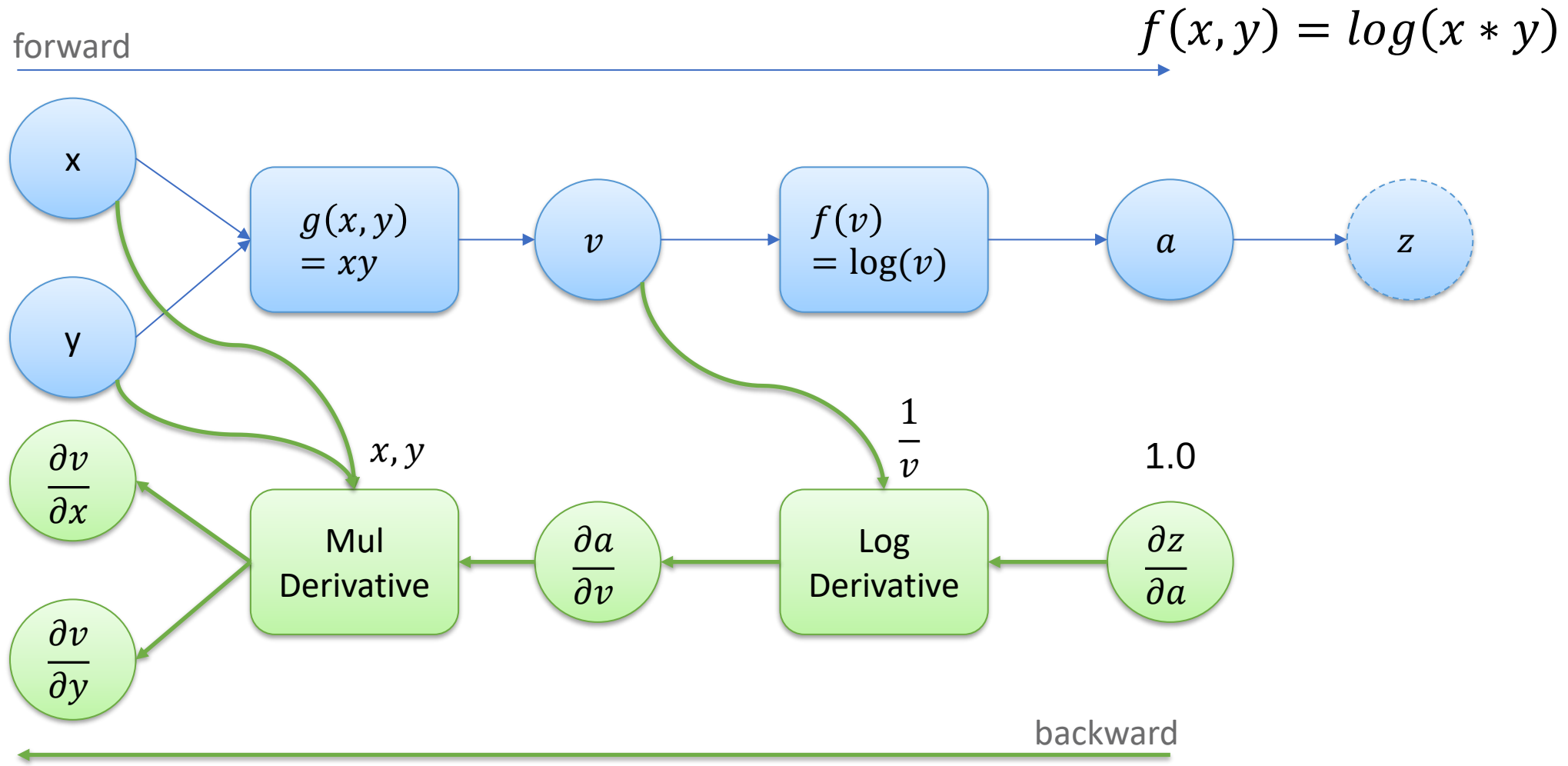
Computational graph: forward and computational graph extended

- $f(x, y) = f(v) = \log(v) = a$
- $\frac{\partial}{\partial v} f(v) = \frac{\partial a}{\partial v} = \frac{1}{v}$



Computational graph: backward pass (gradient)

- $\frac{\partial}{\partial x} f(g(x, y)) = \frac{\partial z}{\partial a} \frac{\partial a}{\partial v} \frac{\partial v}{\partial x} = 1.0 * \frac{1}{v} * y$
- $\frac{\partial}{\partial y} f(g(x, y)) = \frac{\partial z}{\partial a} \frac{\partial a}{\partial v} \frac{\partial v}{\partial y} = 1.0 * \frac{1}{v} * x$



Computational graph

- Overview of PyTorch Autograd Engine: <https://pytorch.org/blog/overview-of-pytorch-autograd-engine/>
- A Gentle Introduction to torch.autograd : https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html
- What is Automatic Differentiation? : https://www.youtube.com/watch?v=wG_nF1awSSY